EGE ENİVERSİTESİ
TİRE KUTSAN MESLEK YÜKSEKOKULU
2025-2026 Eğitim Öğretim Yılı Görsel Programlama Dersi Proje Final Raporu
Danışman: Öğr. Gör. Yeşim AKTAŞ

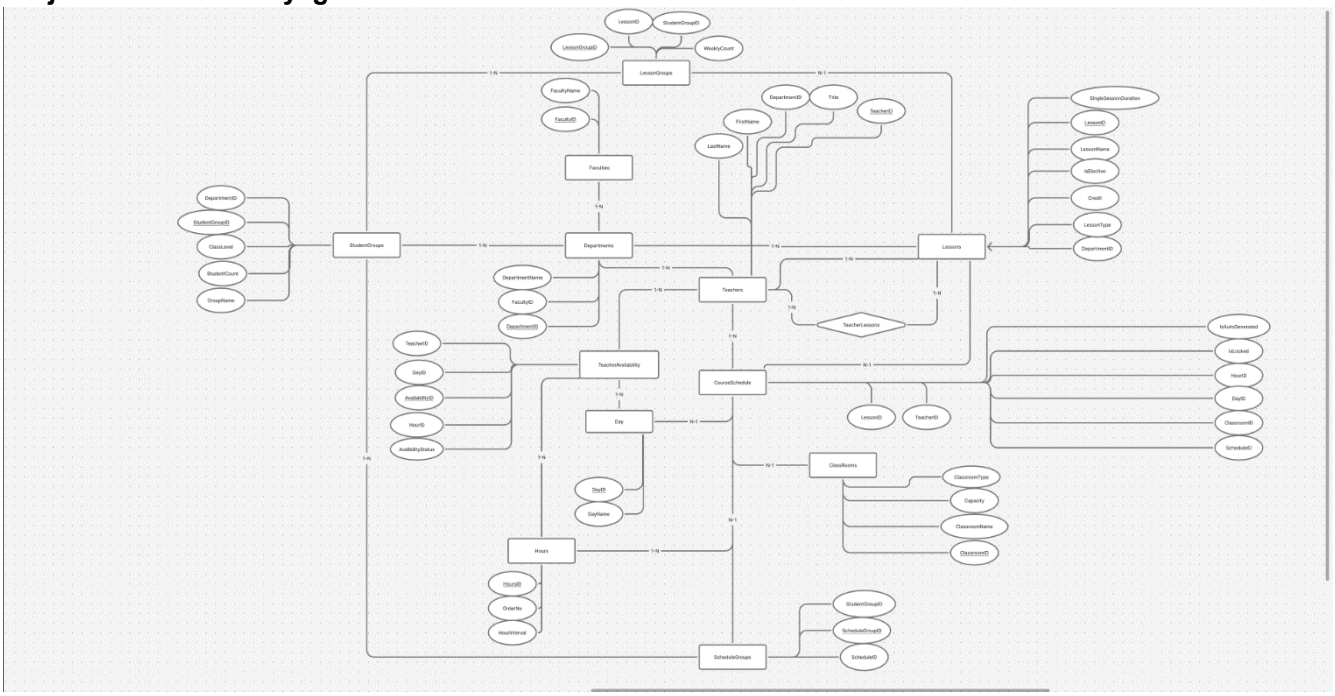
Öğrenci No: 60250000047, 60250000043

Öğrenci Adı Soyadı: Eyüp KARAKAYA, Kağan Kemal ÜRE

Proje Adı: Bölüm Ders Programı Oluşturucu

Projenin Video Linki: Course Schedule - Google Drive

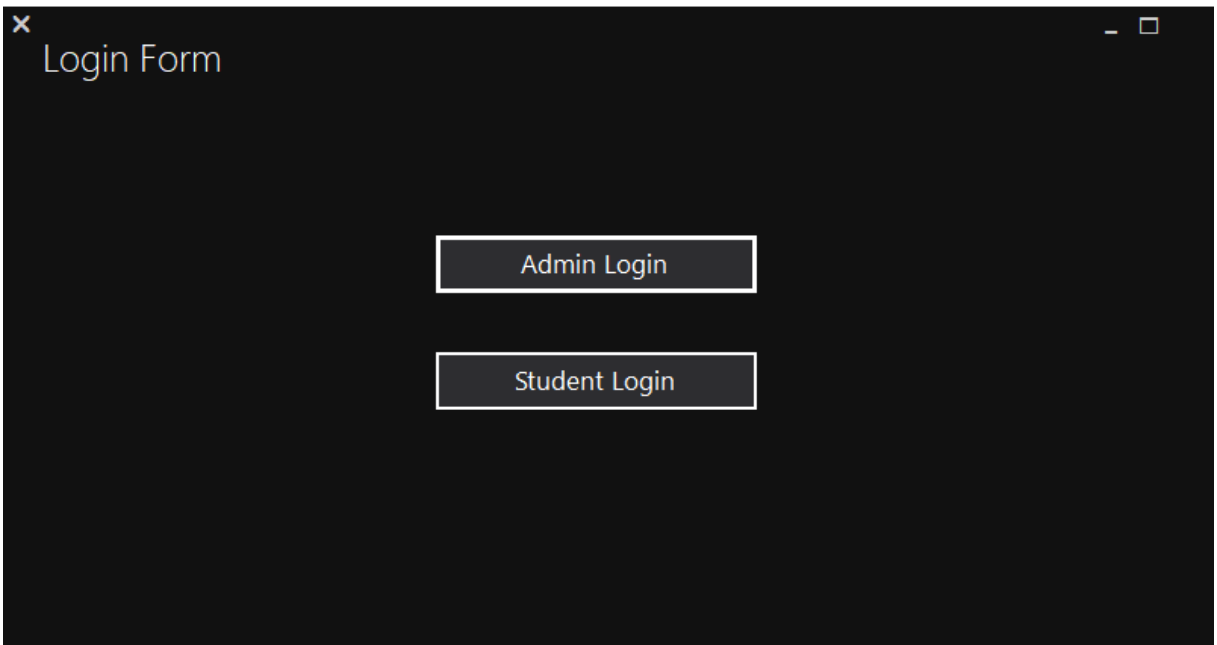
Projenin veri tabanı diyagramı:



Yukarıda bulunan veri tabanı diyagramının okunamaması halinde videoyu paylaştığımız link üzerinden yukarıda bulunan veri tabanı diyagramının da orijinal halini paylaşmış bulunmaktayız. Teşekkürler.

Projenin Ekran Görüntüleri ve Kodları:

LOGİN FORMU EKRAN FOTOĞRAFI:



LOGİN FORMU KODLARI:

```
using MetroFramework.Forms;
using System;
using System.Windows.Forms;

namespace CourseSchedule
{
    public partial class LoginForm : MetroForm
    {
        public LoginForm()
        {
            InitializeComponent();
            this.StartPosition = FormStartPosition.CenterScreen; // Formu tam ortada aç


        }

        private void btnAdmin_Click(object sender, EventArgs e)
        {
            Form1 mainForm = new Form1("Admin");
            this.Hide();
            mainForm.ShowDialog();
            this.Show();
        }

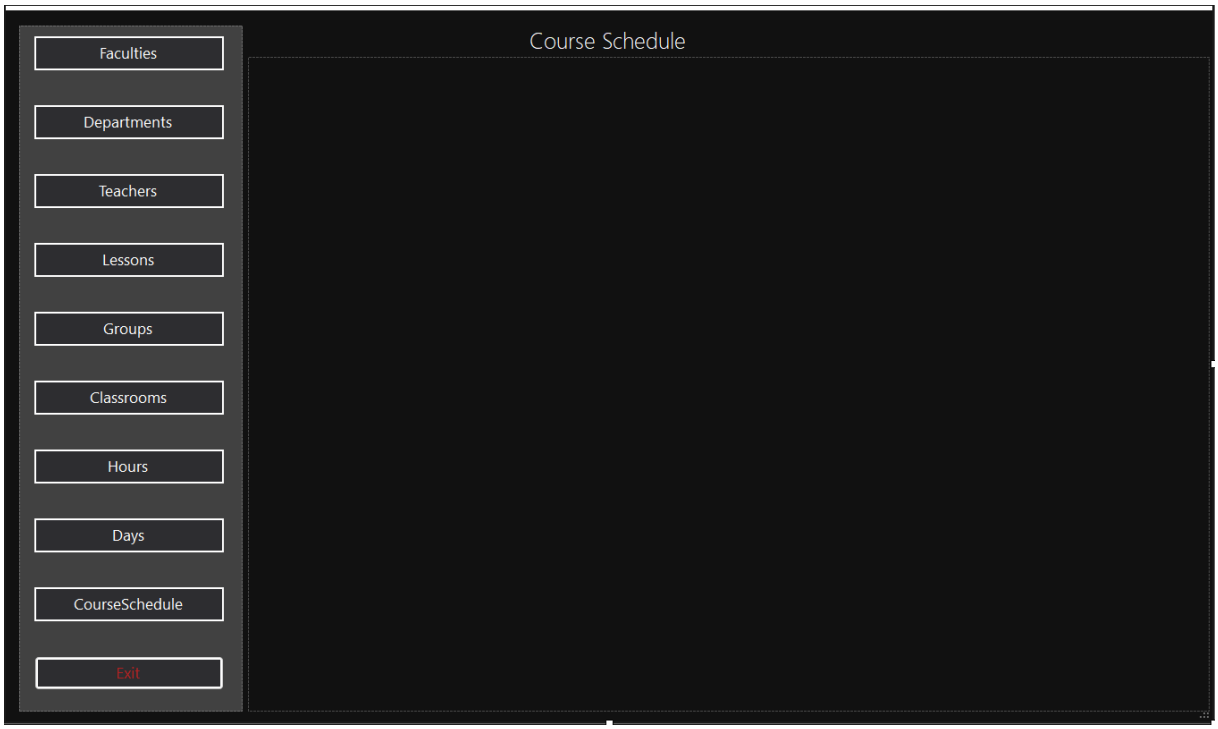
        private void btnStudent_Click(object sender, EventArgs e)
        {
            Form1 mainForm = new Form1("Student");
```

```
            this.Hide();
            mainForm.ShowDialog();
            this.Show();
        }

        private void LoginForm_Load(object sender, EventArgs e)
        {

        }
    }
}
```

FORM1 EKRAN FOTOĞRAFI:



FORM1 KODLARI:

```
using Microsoft.Data.SqlClient;
using MetroFramework.Forms;
using System.Linq;

namespace CourseSchedule
{
    public partial class Form1 : MetroForm
    {
        public static string UserRole = "Admin";

        public Form1(string role)
        {
```

```
    InitializeComponent();
    UserRole = role;

    if (UserRole == "Student")
    {
        Faculties_Button.Visible = false;
        Departments_Button.Visible = false;
        Teachers_Button.Visible = false;
        Lessons_Button.Visible = false;
        Groups_Button.Visible = false;
        Classrooms_Button.Visible = false;
        Hours_Button.Visible = false;
        Days_Button.Visible = false;

        LoadControl(new CourseScheduleControl());
    }
    else
    {

    }
}

private void LoadControl(UserControl uc)
{
    mainPanel.Controls.Clear();
    uc.Dock = DockStyle.Fill;
    mainPanel.Controls.Add(uc);
```

```
}

private void CloseOtherForms()
{
    foreach (Form form in Application.OpenForms)
    {
        if (form.GetType().Name == "FacultyForm" ||
            form.GetType().Name == "DepartmentsForm" ||
            form.GetType().Name == "TeacherForm" ||
            form.GetType().Name == "LessonForm" ||
            form.GetType().Name == "StudentGroupForm" ||
            form.GetType().Name == "ClassroomForm")
        {
            form.Close();
            break;
        }
    }
}
private void Faculties_Button_Click(object sender, EventArgs e)
{
    LoadControl(new FacultiesControl());
}

private void Departments_Button_Click(object sender, EventArgs e)
{
    LoadControl(new DepartmentsControl());
}
```

```
private void Teachers_Button_Click(object sender, EventArgs e)
{
   LoadControl(new TeachresControl());
}
private void Lessons_Button_Click(object sender, EventArgs e)
{
   LoadControl(new LessonsControl());
}

private void Groups_Button_Click(object sender, EventArgs e)
{
   LoadControl(new GroupsControl());
}

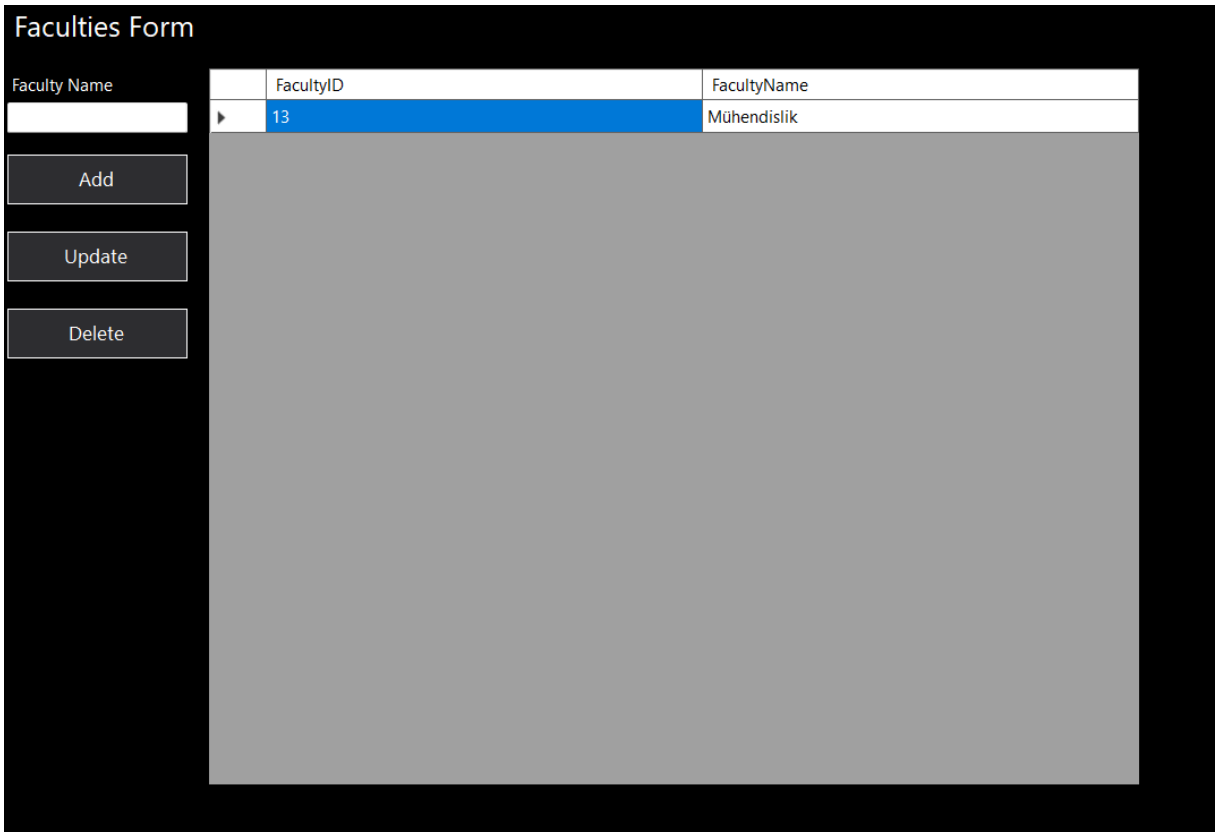
private void Classrooms_Button_Click(object sender, EventArgs e)
{
   LoadControl(new ClassroomsControl());
}

private void CourseSchedule_Button_Click(object sender, EventArgs e)
{
   LoadControl(new CourseScheduleControl());
}
private void Hours_Button_Click(object sender, EventArgs e)
{
   LoadControl(new HourIntervalControl());
}
```

```
        private void Days_Button_Click(object sender, EventArgs e)
        {
            LoadControl(new DaysControl());
        }
        private void Exit_Button_Click(object sender, EventArgs e)
        {
            this.Close();
        }
        private void Form1_Load(object sender, EventArgs e)
        {

        }
    }
}
```

FACULTİES FORMU EKRAN FOTOĞRAFI:



FACULTİES FORMU KODLARI:

```
using MetroFramework.Forms;
using Microsoft.Data.SqlClient;
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Text;
using System.Windows.Forms;
```

```
namespace CourseSchedule
{
    public partial class FacultiesControl : UserControl
    {
        public FacultiesControl()
        {
            InitializeComponent();
            LoadFaculties();
        }

        private void FacultyForm_Load(object sender, EventArgs e)
        {
            LoadFaculties();
        }

        private void LoadFaculties()
        {
            using (var conn = DatabaseHelper.GetConnection())
            {
                conn.Open();
                var cmd = new Microsoft.Data.SqlClient.SqlCommand("select * from Faculties", conn);
                var adapter = new Microsoft.Data.SqlClient.SqlDataAdapter(cmd);
                var table = new DataTable();
                adapter.Fill(table);
                dataGridView1.DataSource = table;
            }
```

```
}

private void Add_Button_Click(object sender, EventArgs e)
{
    if (textBox1.Text == "")
    {
        MessageBox.Show("Please enter a faculty name.");
        return;
    }

    using (var conn = DatabaseHelper.GetConnection())
    {
        conn.Open();
        var cmd = new Microsoft.Data.SqlClient.SqlCommand(
            "insert into Faculties (FacultyName) values (@FacultyName)", conn);
        cmd.Parameters.AddWithValue("@FacultyName", textBox1.Text);
        cmd.ExecuteNonQuery();
        MessageBox.Show("Faculty added successfully.");
        textBox1.Text = "";
        LoadFaculties();
    }
}

private void Delete_Button_Click(object sender, EventArgs e)
{
    if (dataGridView1.SelectedRows.Count == 0)
    {
```

```
            MessageBox.Show("Please select a faculty to delete.");
            return;
        }

        int facultyID = Convert.ToInt32(dataGridView1.SelectedRows[0].Cells["FacultyID"].Value);

        try
        {
            using (var conn = DatabaseHelper.GetConnection())
            {
                conn.Open();
                var cmd = new Microsoft.Data.SqlClient.SqlCommand(
                    "delete from Faculties where FacultyID = @FacultyID", conn);
                cmd.Parameters.AddWithValue("@FacultyID", facultyID);
                cmd.ExecuteNonQuery();
                MessageBox.Show("Faculty deleted successfully.");
                LoadFaculties();
            }
        }
        catch (SqlException ex)
        {
            if (ex.Number == 547)
            {
                MessageBox.Show("Cannot delete this faculty because it has related departments. Delete the departments first.");
            }
            else
```

```
        {
            MessageBox.Show("Error: " + ex.Message);
        }
    }
}


private void Update_Button_Click(object sender, EventArgs e)
{
    if (dataGridView1.SelectedRows.Count == 0)
    {
        MessageBox.Show("Please select a faculty to update.");
        return;
    }

    if (textBox1.Text == "")
    {
        MessageBox.Show("Please enter a new faculty name.");
        return;
    }

    int FacultyID =
Convert.ToInt32(dataGridView1.SelectedRows[0].Cells["FacultyID"].Value);
    string newFacultyName = textBox1.Text;

    using (var conn = DatabaseHelper.GetConnection())
    {
        conn.Open();
```

```
            var cmd = new Microsoft.Data.SqlClient.SqlCommand(
              "update Faculties set FacultyName = @newFacultyName where FacultyID =
@FacultyID", conn);
            cmd.Parameters.AddWithValue("@newFacultyName", newFacultyName);
            cmd.Parameters.AddWithValue("@FacultyID", FacultyID);
            cmd.ExecuteNonQuery();
            MessageBox.Show("Faculty update successfully.");
            textBox1.Text = "";
            LoadFaculties();

        }
    }

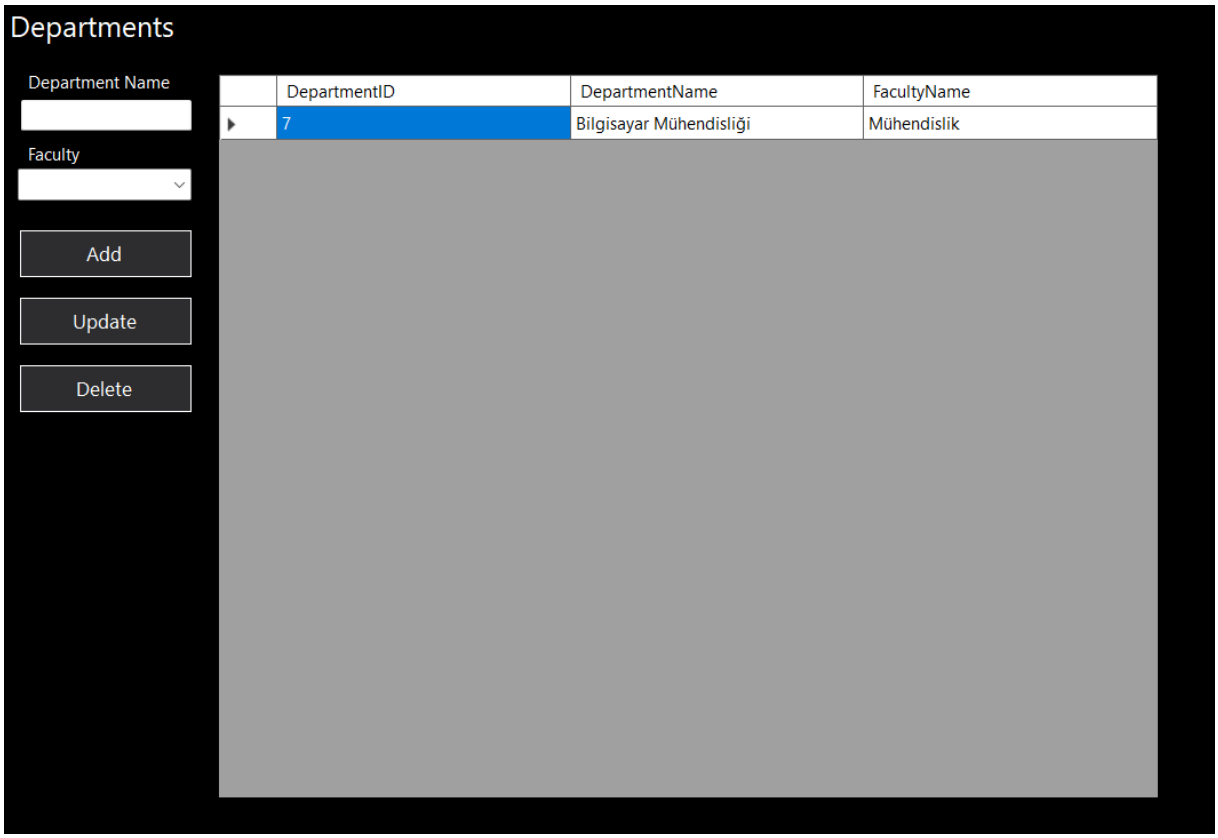
    private void FacultiesControl_Load(object sender, EventArgs e)
    {

    }

    private void FacultiesControl_Load_1(object sender, EventArgs e)
    {

    }
  }
}
```

DEPARTMENTS FORMU EKRAN FOTOĞRAFI:



DEPARTMENTS FORMU KODLARI:

```
using MetroFramework.Forms;
using Microsoft.Data.SqlClient;
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Text;
using System.Windows.Forms;
```

```
namespace CourseSchedule
{
    public partial class DepartmentsControl : UserControl
    {
        public DepartmentsControl()
        {
            InitializeComponent();
            LoadFaculties();
            LoadDepartments();
        }

        private void DepartmentsForm_Load(object sender, EventArgs e)
        {
            LoadFaculties();
            LoadDepartments();
        }

        private void LoadFaculties()
        {
            using (var conn = DatabaseHelper.GetConnection())
            {
                conn.Open();
                var cmd = new Microsoft.Data.SqlClient.SqlCommand("select FacultyID, FacultyName
from Faculties", conn);
                var reader = cmd.ExecuteReader();
                comboBox1.Items.Clear();
                while (reader.Read())
```

```
            {
                comboBox1.Items.Add(new { ID = reader["FacultyID"], Name = reader["FacultyName"].ToString() });
            }
            comboBox1.DisplayMember = "Name";
            comboBox1.ValueMember = "ID";
        }
    }

    private void LoadDepartments()
    {
        using (var conn = DatabaseHelper.GetConnection())
        {
            conn.Open();
            var cmd = new Microsoft.Data.SqlClient.SqlCommand(
                "select d.DepartmentID, d.DepartmentName, f.FacultyName from Departments d join Faculties f on d.FacultyID = f.FacultyID", conn);
            var adapter = new Microsoft.Data.SqlClient.SqlDataAdapter(cmd);
            var table = new DataTable();
            adapter.Fill(table);
            dataGridView1.DataSource = table;
        }

    }

    private void Add_Button_Click(object sender, EventArgs e)
    {
        if (textBox1.Text == "" || comboBox1.SelectedItem == null)
```

```
        {
            MessageBox.Show("Please enter department name and select a faculty.");
            return;
        }

        dynamic selectedFaculty = comboBox1.SelectedItem;
        int FacultyID = selectedFaculty.ID;

        using (var conn = DatabaseHelper.GetConnection())
        {
            conn.Open();
            var cmd = new Microsoft.Data.SqlClient.SqlCommand(
                "insert into Departments (DepartmentName, FacultyID) values (@DepartmentName, @FacultyId)", conn);
            cmd.Parameters.AddWithValue("@DepartmentName", textBox1.Text);
            cmd.Parameters.AddWithValue("@FacultyId", FacultyID);
            cmd.ExecuteNonQuery();
            MessageBox.Show("Department added successfully.");
            textBox1.Text = "";
            comboBox1.SelectedIndex = -1;
            LoadDepartments();
        }
    }

    private void Update_Button_Click(object sender, EventArgs e)
    {
        if (dataGridView1.SelectedRows.Count == 0)
        {
```

```
            MessageBox.Show("Please select a faculty to update.");
            return;
        }

        if (textBox1.Text == "" || comboBox1.SelectedItem == null)
        {
            MessageBox.Show("Please enter a new faculty name.");
            return;
        }

        int DepartmentID = Convert.ToInt32(dataGridView1.SelectedRows[0].Cells["DepartmentID"].Value);
        string newDepartmentName = textBox1.Text;
        dynamic selectedFaculty = comboBox1.SelectedItem;
        int FacultyID = selectedFaculty.ID;

        using (var conn = DatabaseHelper.GetConnection())
        {
            conn.Open();
            var cmd = new Microsoft.Data.SqlClient.SqlCommand(
              "update Departments set DepartmentName = @newDepartmentName where DepartmentID = @DepartmentID", conn);
            cmd.Parameters.AddWithValue("@newDepartmentName", newDepartmentName);
            cmd.Parameters.AddWithValue("@FacultyID", FacultyID);
            cmd.Parameters.AddWithValue("@DepartmentID", DepartmentID);
            cmd.ExecuteNonQuery();
            MessageBox.Show("Department update successfully.");
            textBox1.Text = "";
```

```
            comboBox1.SelectedIndex = -1;
            LoadDepartments();

        }
    }

    private void Delete_Button_Click(object sender, EventArgs e)
    {
        if (dataGridView1.SelectedRows.Count == 0)
        {
            MessageBox.Show("Please select a department to delete.");
            return;
        }

        int departmentID =
Convert.ToInt32(dataGridView1.SelectedRows[0].Cells["DepartmentID"].Value);

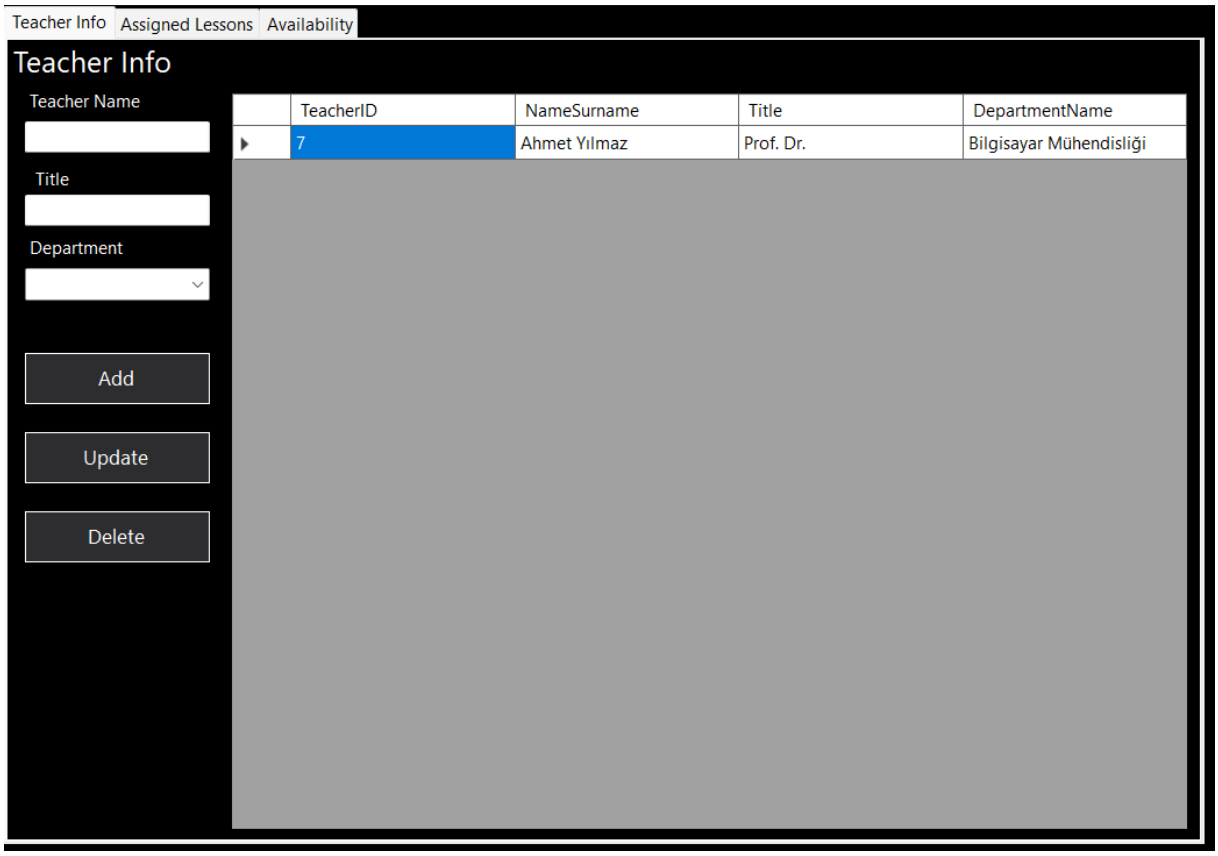
        try
        {
            using (var conn = DatabaseHelper.GetConnection())
            {
                conn.Open();
                var cmd = new Microsoft.Data.SqlClient.SqlCommand(
                "delete from Departments where DepartmentID = @DepartmentID", conn);
                cmd.Parameters.AddWithValue("@DepartmentID", departmentID);
                cmd.ExecuteNonQuery();
                MessageBox.Show("Department deleted successfully.");
                LoadDepartments();
```

```
            }
        }
        catch (SqlException ex)
        {
            if (ex.Number == 547)
            {
                MessageBox.Show("Cannot delete! This record has related data in other tables.");
            }
            else
            {
                MessageBox.Show("Error: " + ex.Message);
            }
        }
    }

    private void DepartmentsControl_Load(object sender, EventArgs e)
    {

    }
  }
}
```

TEACHERS FORMU EKRAN FOTOĞRAFI:



TEACHERS FORMU KODLARI:

```
using Microsoft.Data.SqlClient;
using System;
using System.Data;
using System.Windows.Forms;

namespace CourseSchedule
{
    public partial class TeachresControl : UserControl
    {
```

```
        public TeachresControl()
        {
            InitializeComponent();

            // Yüklemeler
            LoadDepartments();
            LoadTeachers();
            LoadTeachersCombo(); // Hem Tab 2 hem Tab 3'teki combobox'ları doldurur
            LoadLessonsCombo();
            LoadTeacherLessonsGrid();

            // Tab 3 Özel Yüklemeler
            LoadDaysCombo();
            LoadHoursCombo();
            LoadAvailabilityGrid();
        }

        // ========================================================================
        // TAB 1: TEACHER INFO
        // ========================================================================

        private void LoadDepartments()
        {
            using (var conn = DatabaseHelper.GetConnection())
            {
                conn.Open();
                var cmd = new SqlCommand("select DepartmentID, DepartmentName from Departments",
conn);
```

```
            var reader = cmd.ExecuteReader();
            comboBox1.Items.Clear();
            while (reader.Read())
            {
                comboBox1.Items.Add(new { ID = reader["DepartmentID"], Name =
reader["DepartmentName"].ToString() });
            }
            reader.Close();
        }
        comboBox1.DisplayMember = "Name";
        comboBox1.ValueMember = "ID";
    }

    private void LoadTeachers()
    {
        using (var conn = DatabaseHelper.GetConnection())
        {
            conn.Open();
            var cmd = new SqlCommand(@"
                select t.TeacherID, t.NameSurname, t.Title, d.DepartmentName
                from Teachers t
                join Departments d on t.DepartmentID = d.DepartmentID", conn);
            var adapter = new SqlDataAdapter(cmd);
            var table = new DataTable();
            adapter.Fill(table);
            dataGridView1.DataSource = table;
        }
    }
```

```
        private void Add_Button_Click(object sender, EventArgs e)
        {
            if (string.IsNullOrWhiteSpace(textBox1.Text) || string.IsNullOrWhiteSpace(textBox2.Text) ||
comboBox1.SelectedItem == null)
            {
                MessageBox.Show("Please fill all fields!");
                return;
            }

            dynamic selectedDept = comboBox1.SelectedItem;
            int deptID = selectedDept.ID;

            using (var conn = DatabaseHelper.GetConnection())
            {
                conn.Open();
                var cmd = new SqlCommand("insert into Teachers (NameSurname, Title, DepartmentID)
values (@Name, @Title, @DeptID)", conn);
                cmd.Parameters.AddWithValue("@Name", textBox1.Text);
                cmd.Parameters.AddWithValue("@Title", textBox2.Text);
                cmd.Parameters.AddWithValue("@DeptID", deptID);
                cmd.ExecuteNonQuery();

                MessageBox.Show("Teacher added successfully!");
                textBox1.Clear();
                textBox2.Clear();
                comboBox1.SelectedIndex = -1;
```

```
            LoadTeachers();
            LoadTeachersCombo();
        }
    }

    private void Update_Button_Click(object sender, EventArgs e)
    {
        if (dataGridView1.SelectedRows.Count == 0)
        {
            MessageBox.Show("Please select a teacher to update.");
            return;
        }

        if (string.IsNullOrWhiteSpace(textBox1.Text) || string.IsNullOrWhiteSpace(textBox2.Text) ||
comboBox1.SelectedItem == null)
        {
            MessageBox.Show("Please fill all fields!");
            return;
        }

        int teacherID = Convert.ToInt32(dataGridView1.SelectedRows[0].Cells["TeacherID"].Value);
        dynamic selectedDept = comboBox1.SelectedItem;
        int deptID = selectedDept.ID;

        using (var conn = DatabaseHelper.GetConnection())
        {
            conn.Open();
```

```
            var cmd = new SqlCommand("update Teachers set NameSurname = @Name, Title =
@Title, DepartmentID = @DeptID where TeacherID = @TeacherID", conn);
            cmd.Parameters.AddWithValue("@Name", textBox1.Text);
            cmd.Parameters.AddWithValue("@Title", textBox2.Text);
            cmd.Parameters.AddWithValue("@DeptID", deptID);
            cmd.Parameters.AddWithValue("@TeacherID", teacherID);
            cmd.ExecuteNonQuery();

            MessageBox.Show("Teacher updated successfully.");
            textBox1.Clear();
            textBox2.Clear();
            comboBox1.SelectedIndex = -1;

            LoadTeachers();
            LoadTeachersCombo();
        }
    }

    private void Delete_Button_Click(object sender, EventArgs e)
    {
        if (dataGridView1.SelectedRows.Count == 0)
        {
            MessageBox.Show("Please select a teacher to delete.");
            return;
        }

        int teacherID = Convert.ToInt32(dataGridView1.SelectedRows[0].Cells["TeacherID"].Value);
```

```
        try
        {
            using (var conn = DatabaseHelper.GetConnection())
            {
                conn.Open();
                var cmd = new SqlCommand("delete from Teachers where TeacherID = @TeacherID",
conn);
                cmd.Parameters.AddWithValue("@TeacherID", teacherID);
                cmd.ExecuteNonQuery();


                MessageBox.Show("Teacher deleted successfully.");
                LoadTeachers();
                LoadTeachersCombo();
            }
        }
        catch (SqlException ex)
        {
            if (ex.Number == 547)
            {
                MessageBox.Show("Cannot delete! This teacher has related data.");
            }
            else
            {
                MessageBox.Show("Error: " + ex.Message);
            }
        }
    }
```

```
// ======================================================================
// ORTAK COMBOBOX YÜKLEYİCİ (Hem Tab 2 hem Tab 3 için)
// ======================================================================

private void LoadTeachersCombo()
{
    using (var conn = DatabaseHelper.GetConnection())
    {
        conn.Open();
        var cmd = new SqlCommand("select TeacherID, NameSurname from Teachers", conn);
        var reader = cmd.ExecuteReader();

        cbTeacher.Items.Clear();
        cbAvailTeacher.Items.Clear();

        while (reader.Read())
        {
            var item = new { ID = reader["TeacherID"], Name = reader["NameSurname"].ToString() };
            cbTeacher.Items.Add(item);
            cbAvailTeacher.Items.Add(item);
        }
        reader.Close();
    }
    cbTeacher.DisplayMember = "Name";
    cbTeacher.ValueMember = "ID";

    cbAvailTeacher.DisplayMember = "Name";
```

```
            cbAvailTeacher.ValueMember = "ID";
        }

        // ======================================================================
        // TAB 2: ASSIGNED LESSONS
        // ======================================================================

        private void LoadLessonsCombo()
        {
            using (var conn = DatabaseHelper.GetConnection())
            {
                conn.Open();
                var cmd = new SqlCommand("select LessonID, LessonName from Lessons", conn);
                var reader = cmd.ExecuteReader();
                cbLesson.Items.Clear();
                while (reader.Read())
                {
                    cbLesson.Items.Add(new { ID = reader["LessonID"], Name =
reader["LessonName"].ToString() });
                }
                reader.Close();
            }
            cbLesson.DisplayMember = "Name";
            cbLesson.ValueMember = "ID";
        }

        private void LoadTeacherLessonsGrid()
        {
```

```
        using (var conn = DatabaseHelper.GetConnection())
        {
            conn.Open();
            var cmd = new SqlCommand(@"
                select tl.TeacherLessonID, t.NameSurname as [Teacher Name], l.LessonName as
[Lesson Name]
                from TeacherLessons tl
                join Teachers t on tl.TeacherID = t.TeacherID
                join Lessons l on tl.LessonID = l.LessonID", conn);
            var adapter = new SqlDataAdapter(cmd);
            var table = new DataTable();
            adapter.Fill(table);
            dgvTeacherLessons.DataSource = table;
        }
    }

    private void btnAddTeacherLesson_Click(object sender, EventArgs e)
    {
        if (cbTeacher.SelectedItem == null || cbLesson.SelectedItem == null)
        {
            MessageBox.Show("Please select both a Teacher and a Lesson!");
            return;
        }

        dynamic selectedTeacher = cbTeacher.SelectedItem;
        dynamic selectedLesson = cbLesson.SelectedItem;

        using (var conn = DatabaseHelper.GetConnection())
```

```
        {
            conn.Open();
            var cmd = new SqlCommand("insert into TeacherLessons (TeacherID, LessonID) values
(@TeacherID, @LessonID)", conn);
            cmd.Parameters.AddWithValue("@TeacherID", selectedTeacher.ID);
            cmd.Parameters.AddWithValue("@LessonID", selectedLesson.ID);
            cmd.ExecuteNonQuery();

            MessageBox.Show("Lesson assigned to teacher successfully!");
            cbTeacher.SelectedIndex = -1;
            cbLesson.SelectedIndex = -1;

            LoadTeacherLessonsGrid();
        }
    }

    private void btnDeleteTeacherLesson_Click(object sender, EventArgs e)
    {
        if (dgvTeacherLessons.SelectedRows.Count == 0)
        {
            MessageBox.Show("Please select an assignment to remove.");
            return;
        }

        int teacherLessonID =
Convert.ToInt32(dgvTeacherLessons.SelectedRows[0].Cells["TeacherLessonID"].Value);

        try
```

```
        {
            using (var conn = DatabaseHelper.GetConnection())
            {
                conn.Open();
                var cmd = new SqlCommand("delete from TeacherLessons where TeacherLessonID =
@ID", conn);
                cmd.Parameters.AddWithValue("@ID", teacherLessonID);
                cmd.ExecuteNonQuery();

                MessageBox.Show("Assignment removed successfully.");
                LoadTeacherLessonsGrid();
            }
        }
        catch (SqlException ex)
        {
            MessageBox.Show("Error: " + ex.Message);
        }
    }

    // ========================================================================
    // TAB 3: AVAILABILITY (Müsaitlik)
    // ========================================================================

    private void LoadDaysCombo()
    {
        using (var conn = DatabaseHelper.GetConnection())
        {
            conn.Open();
```

```
            var cmd = new SqlCommand("select DayID, DayTitle from tbl_Days", conn);
            var reader = cmd.ExecuteReader();
            cbAvailDay.Items.Clear();
            while (reader.Read())
            {
                cbAvailDay.Items.Add(new { ID = reader["DayID"], Name = reader["DayTitle"].ToString()
});
            }
            reader.Close();
        }
        cbAvailDay.DisplayMember = "Name";
        cbAvailDay.ValueMember = "ID";
    }

    private void LoadHoursCombo()
    {
        using (var conn = DatabaseHelper.GetConnection())
        {
            conn.Open();
            var cmd = new SqlCommand("select HourID, HourInterval from tbl_Hours order by
OrderNo", conn);
            var reader = cmd.ExecuteReader();
            cbAvailHour.Items.Clear();
            while (reader.Read())
            {
                cbAvailHour.Items.Add(new { ID = reader["HourID"], Name =
reader["HourInterval"].ToString() });
            }
            reader.Close();
```

```
    }
    cbAvailHour.DisplayMember = "Name";
    cbAvailHour.ValueMember = "ID";
}

private void LoadAvailabilityGrid()
{
    using (var conn = DatabaseHelper.GetConnection())
    {
        conn.Open();
        var cmd = new SqlCommand(@"
            select
                ta.UnID,
                t.NameSurname as [Teacher],
                d.DayTitle as [Day],
                h.HourInterval as [Hour],
                case when ta.AvailabilityStatus = 1 then 'Yes' else 'No' end as [Available]
            from TeacherAvailability ta
            join Teachers t on ta.TeacherID = t.TeacherID
            join tbl_Days d on ta.DayID = d.DayID
            join tbl_Hours h on ta.HourID = h.HourID", conn);
        var adapter = new SqlDataAdapter(cmd);
        var table = new DataTable();
        adapter.Fill(table);
        dgvAvailability.DataSource = table;
    }
}
```

```
    private void btnAddAvail_Click(object sender, EventArgs e)
    {
        if (cbAvailTeacher.SelectedItem == null || cbAvailDay.SelectedItem == null ||
cbAvailHour.SelectedItem == null)
        {
            MessageBox.Show("Please select Teacher, Day and Hour!");
            return;
        }

        dynamic selectedTeacher = cbAvailTeacher.SelectedItem;
        dynamic selectedDay = cbAvailDay.SelectedItem;
        dynamic selectedHour = cbAvailHour.SelectedItem;
        bool status = chkIsAvailable.Checked;

        using (var conn = DatabaseHelper.GetConnection())
        {
            conn.Open();
            var cmd = new SqlCommand("insert into TeacherAvailability (TeacherID, DayID, HourID,
AvailabilityStatus) values (@TeacherID, @DayID, @HourID, @Status)", conn);
            cmd.Parameters.AddWithValue("@TeacherID", selectedTeacher.ID);
            cmd.Parameters.AddWithValue("@DayID", selectedDay.ID);
            cmd.Parameters.AddWithValue("@HourID", selectedHour.ID);
            cmd.Parameters.AddWithValue("@Status", status);
            cmd.ExecuteNonQuery();

            MessageBox.Show("Availability status set successfully!");
            cbAvailDay.SelectedIndex = -1;
```

```
        cbAvailHour.SelectedIndex = -1;
        chkIsAvailable.Checked = false;

        LoadAvailabilityGrid();
    }
}

private void btnDeleteAvail_Click(object sender, EventArgs e)
{
    if (dgvAvailability.SelectedRows.Count == 0)
    {
        MessageBox.Show("Please select a status to remove.");
        return;
    }

    int unID = Convert.ToInt32(dgvAvailability.SelectedRows[0].Cells["UnID"].Value);

    try
    {
        using (var conn = DatabaseHelper.GetConnection())
        {
            conn.Open();
            var cmd = new SqlCommand("delete from TeacherAvailability where UnID = @ID", conn);
            cmd.Parameters.AddWithValue("@ID", unID);
            cmd.ExecuteNonQuery();

            MessageBox.Show("Status removed successfully.");
```

```
                LoadAvailabilityGrid();
            }
        }
        catch (SqlException ex)
        {
            MessageBox.Show("Error: " + ex.Message);
        }
    }
    private void label10_Click(object sender, EventArgs e)
    {

    }

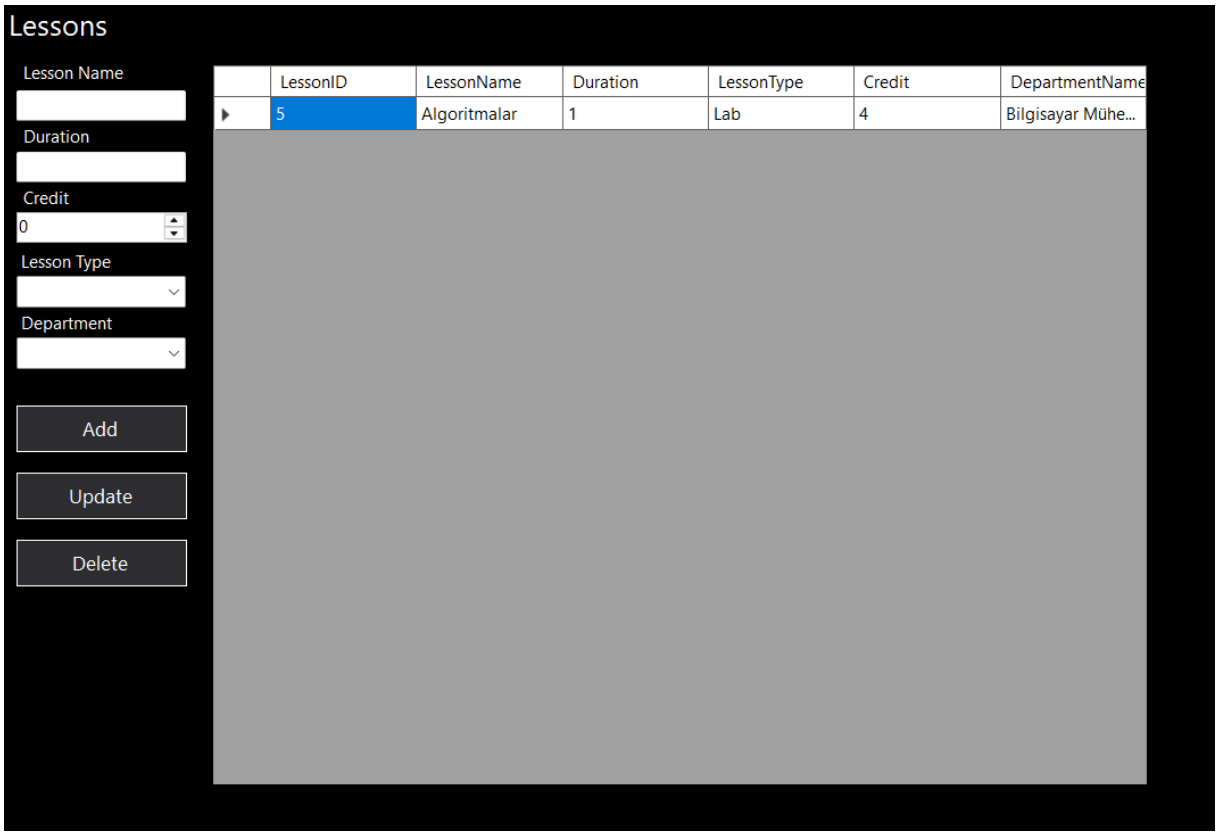
    private void label8_Click(object sender, EventArgs e)
    {

    }

    private void tabPage1_Click(object sender, EventArgs e)
    {

    }
  }
}
```

LESSONS FORMU EKRAN FOTOĞRAFI:



LESSONS FORMU KODLARI:

```
using MetroFramework.Forms;
using Microsoft.Data.SqlClient;
using System;
using System.Data;
using System.Windows.Forms;

namespace CourseSchedule
{
    public partial class LessonsControl : UserControl
    {
```

```
public LessonsControl()
{
    InitializeComponent();
    LoadLessonTypes();
    LoadDepartments();
    LoadLessons();
}

private void Lesson_Load(object sender, EventArgs e)
{
    LoadLessonTypes();
    LoadDepartments();
    LoadLessons();
}

private void LoadLessonTypes()
{
    LessonCombo.Items.Clear();
    LessonCombo.Items.Add("Teorik");
    LessonCombo.Items.Add("Uygulama");
    LessonCombo.Items.Add("Lab");
}

private void LoadDepartments()
{
    using (var conn = DatabaseHelper.GetConnection())
    {
```

```
            conn.Open();
            var cmd = new SqlCommand("select DepartmentID, DepartmentName from Departments",
conn);
            var reader = cmd.ExecuteReader();
            DepartmentCombo.Items.Clear();


            while (reader.Read())
            {
                DepartmentCombo.Items.Add(new { ID = reader["DepartmentID"], Name =
reader["DepartmentName"].ToString() });
            }
            reader.Close();
        }


        DepartmentCombo.DisplayMember = "Name";
        DepartmentCombo.ValueMember = "ID";
    }


    private void LoadLessons()
    {
        using (var conn = DatabaseHelper.GetConnection())
        {
            conn.Open();
            var cmd = new SqlCommand(
                "select l.LessonID, l.LessonName, l.Duration, l.LessonType, l.Credit, d.DepartmentName
from Lessons l join Departments d on l.DepartmentID = d.DepartmentID", conn);
            var adapter = new SqlDataAdapter(cmd);
            var table = new DataTable();
            adapter.Fill(table);
```

```
            dataGridView1.DataSource = table;
        }
    }

    private void Add_Button_Click(object sender, EventArgs e)
    {
        if (LessonName.Text == "" || Duration.Text == "" || DepartmentCombo.SelectedItem == null ||
LessonCombo.SelectedItem == null)
        {
            MessageBox.Show("Please fill all fields!");
            return;
        }

        dynamic selectedDepartment = DepartmentCombo.SelectedItem;
        int DepartmentID = selectedDepartment.ID;
        string lessonType = LessonCombo.SelectedItem?.ToString() ?? "";
        int credit = (int)CreditNumberic.Value;
        int duration = Convert.ToInt32(Duration.Text);

        using (var conn = DatabaseHelper.GetConnection())
        {
            conn.Open();
            var cmd = new SqlCommand(
                "insert into Lessons (LessonName, Duration, LessonType, Credit, DepartmentID) values
(@LessonName, @Duration, @LessonType, @Credit, @DepartmentID)", conn);

            cmd.Parameters.AddWithValue("@LessonName", LessonName.Text);
            cmd.Parameters.AddWithValue("@Duration", duration);
```

```
            cmd.Parameters.AddWithValue("@LessonType", lessonType);
            cmd.Parameters.AddWithValue("@Credit", credit);
            cmd.Parameters.AddWithValue("@DepartmentID", DepartmentID);

            cmd.ExecuteNonQuery();
            MessageBox.Show("Lesson added successfully!");

            LessonName.Text = "";
            Duration.Text = "";
            CreditNumberic.Value = 0;
            DepartmentCombo.SelectedIndex = -1;
            LessonCombo.SelectedIndex = -1;

            LoadLessons();
        }
    }

    private void Update_Button_Click(object sender, EventArgs e)
    {
        if (dataGridView1.SelectedRows.Count == 0)
        {
            MessageBox.Show("Please select a lesson to update.");
            return;
        }

        if (LessonName.Text == "" || Duration.Text == "" || DepartmentCombo.SelectedItem == null ||
LessonCombo.SelectedItem == null)
        {
```

```
            MessageBox.Show("Please fill all fields!");
            return;
        }

        int LessonID = Convert.ToInt32(dataGridView1.SelectedRows[0].Cells["LessonID"].Value);
        dynamic selectedDepartment = DepartmentCombo.SelectedItem;
        int DepartmentID = selectedDepartment.ID;
        int duration = Convert.ToInt32(Duration.Text);

        using (var conn = DatabaseHelper.GetConnection())
        {
            conn.Open();
            var cmd = new SqlCommand(
                "update Lessons set LessonName = @LessonName, Duration = @Duration, LessonType = @LessonType, Credit = @Credit, DepartmentID = @DepartmentID where LessonID = @LessonID", conn);
            cmd.Parameters.AddWithValue("@LessonName", LessonName.Text);
            cmd.Parameters.AddWithValue("@Duration", duration);
            cmd.Parameters.AddWithValue("@LessonType", LessonCombo.SelectedItem.ToString());
            cmd.Parameters.AddWithValue("@Credit", (int)CreditNumberic.Value);
            cmd.Parameters.AddWithValue("@DepartmentID", DepartmentID);
            cmd.Parameters.AddWithValue("@LessonID", LessonID);
            cmd.ExecuteNonQuery();
            MessageBox.Show("Lesson updated successfully.");

            LessonName.Text = "";
            Duration.Text = "";
            CreditNumberic.Value = 0;
```

```
            DepartmentCombo.SelectedIndex = -1;
            LessonCombo.SelectedIndex = -1;
            LoadLessons();
        }
    }

    private void Delete_Button_Click(object sender, EventArgs e)
    {
        if (dataGridView1.SelectedRows.Count == 0)
        {
            MessageBox.Show("Please select a lesson to delete.");
            return;
        }

        int lessonID = Convert.ToInt32(dataGridView1.SelectedRows[0].Cells["LessonID"].Value);

        try
        {
            using (var conn = DatabaseHelper.GetConnection())
            {
                conn.Open();
                var cmd = new SqlCommand("delete from Lessons where LessonID = @LessonID",
conn);
                cmd.Parameters.AddWithValue("@LessonID", lessonID);
                cmd.ExecuteNonQuery();

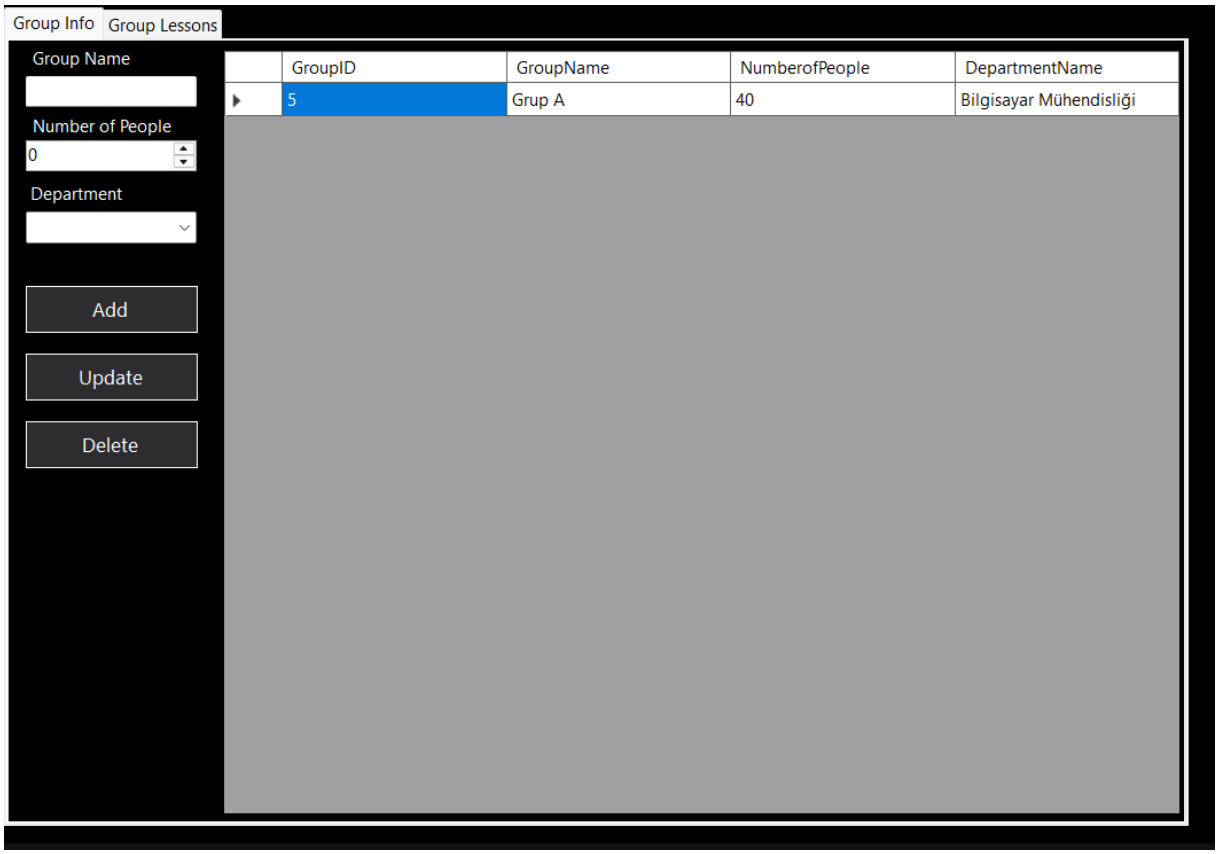
                MessageBox.Show("Lesson deleted successfully.");
                LoadLessons(); // Senin gridi yenileme metodun
```

```
            }
        }
        catch (SqlException ex)
        {
            if (ex.Number == 547)
            {
                MessageBox.Show("Cannot delete! This lesson is already assigned to a teacher or a group. Please remove those assignments first.");
            }
            else
            {
                MessageBox.Show("Error: " + ex.Message);
            }
        }
    }

    private void LessonsControl_Load(object sender, EventArgs e)
    {

    }
  }
}
```

GROUPS FORMU EKRAN FOTOĞRAFI:



GROUPS FORMU KODLARI:

```
using Microsoft.Data.SqlClient;
using System;
using System.Data;
using System.Windows.Forms;

namespace CourseSchedule
{
    public partial class GroupsControl : UserControl
    {
        public GroupsControl()
```

```
        {
            InitializeComponent();

            // Tab 1 (Group Info) Yüklemeleri
            LoadDepartments();
            LoadGroups();

            // Tab 2 (Group Lessons) Yüklemeleri
            LoadGroupsCombo();
            LoadLessonsCombo();
            LoadGroupLessonsGrid();
        }

        // ======================================================================
        // TAB 1: GROUP INFO
        // ======================================================================

        private void LoadDepartments()
        {
            using (var conn = DatabaseHelper.GetConnection())
            {
                conn.Open();
                var cmd = new SqlCommand("select DepartmentID, DepartmentName from Departments",
conn);
                var reader = cmd.ExecuteReader();
                DepartmentCombo.Items.Clear();
                while (reader.Read())
                {
```

```
                DepartmentCombo.Items.Add(new { ID = reader["DepartmentID"], Name =
reader["DepartmentName"].ToString() });
            }
            reader.Close();
        }
        DepartmentCombo.DisplayMember = "Name";
        DepartmentCombo.ValueMember = "ID";
    }

    private void LoadGroups()
    {
        using (var conn = DatabaseHelper.GetConnection())
        {
            conn.Open();
            var cmd = new SqlCommand(@"
                select g.GroupID, g.GroupName, g.NumberofPeople, d.DepartmentName
                from StudentsGroups g
                join Departments d on g.DepartmentID = d.DepartmentID", conn);
            var adapter = new SqlDataAdapter(cmd);
            var table = new DataTable();
            adapter.Fill(table);
            dataGridView1.DataSource = table;
        }
    }

    private void Add_Button_Click(object sender, EventArgs e)
    {
        if (GroupName.Text == "" || DepartmentCombo.SelectedItem == null)
```

```
        {
            MessageBox.Show("Please fill all fields!");
            return;
        }

        dynamic selectedDepartment = DepartmentCombo.SelectedItem;
        int departmentID = selectedDepartment.ID;
        int numberOfPeople = (int)NumberofPeopleNumeric.Value;

        using (var conn = DatabaseHelper.GetConnection())
        {
            conn.Open();
            var cmd = new SqlCommand("insert into StudentsGroups (GroupName, NumberofPeople,
DepartmentID) values (@GroupName, @NumberofPeople, @DepartmentID)", conn);
            cmd.Parameters.AddWithValue("@GroupName", GroupName.Text);
            cmd.Parameters.AddWithValue("@NumberofPeople", numberOfPeople);
            cmd.Parameters.AddWithValue("@DepartmentID", departmentID);
            cmd.ExecuteNonQuery();

            MessageBox.Show("Group added successfully!");
            GroupName.Text = "";
            NumberofPeopleNumeric.Value = 0;
            DepartmentCombo.SelectedIndex = -1;

            LoadGroups();
            LoadGroupsCombo();
        }
    }
```

```
    private void Update_Button_Click(object sender, EventArgs e)
    {
        if (dataGridView1.SelectedRows.Count == 0)
        {
            MessageBox.Show("Please select a group to update.");
            return;
        }

        if (GroupName.Text == "" || DepartmentCombo.SelectedItem == null)
        {
            MessageBox.Show("Please fill all fields!");
            return;
        }

        int groupID = Convert.ToInt32(dataGridView1.SelectedRows[0].Cells["GroupID"].Value);
        dynamic selectedDepartment = DepartmentCombo.SelectedItem;
        int departmentID = selectedDepartment.ID;

        using (var conn = DatabaseHelper.GetConnection())
        {
            conn.Open();
            var cmd = new SqlCommand("update StudentsGroups set GroupName = @GroupName,
NumberofPeople = @NumberofPeople, DepartmentID = @DepartmentID where GroupID =
@GroupID", conn);
            cmd.Parameters.AddWithValue("@GroupName", GroupName.Text);
            cmd.Parameters.AddWithValue("@NumberofPeople", (int)NumberofPeopleNumeric.Value);
            cmd.Parameters.AddWithValue("@DepartmentID", departmentID);
```

```
            cmd.Parameters.AddWithValue("@GroupID", groupID);
            cmd.ExecuteNonQuery();

            MessageBox.Show("Group updated successfully.");
            GroupName.Text = "";
            NumberofPeopleNumeric.Value = 0;
            DepartmentCombo.SelectedIndex = -1;

            LoadGroups();
            LoadGroupsCombo();
        }
    }

    private void Delete_Button_Click(object sender, EventArgs e)
    {
        if (dataGridView1.SelectedRows.Count == 0)
        {
            MessageBox.Show("Please select a group to delete.");
            return;
        }

        int groupID = Convert.ToInt32(dataGridView1.SelectedRows[0].Cells["GroupID"].Value);

        try
        {
            using (var conn = DatabaseHelper.GetConnection())
            {
```

```
                conn.Open();
                var cmd = new SqlCommand("delete from StudentsGroups where GroupID = @GroupID",
conn);
                cmd.Parameters.AddWithValue("@GroupID", groupID);
                cmd.ExecuteNonQuery();

                MessageBox.Show("Group deleted successfully.");
                LoadGroups();
                LoadGroupsCombo();
            }
        }
        catch (SqlException ex)
        {
            if (ex.Number == 547)
            {
                MessageBox.Show("Cannot delete! This group has related data.");
            }
            else
            {
                MessageBox.Show("Error: " + ex.Message);
            }
        }
    }

    // ======================================================================
    // TAB 2: GROUP LESSONS
    // ======================================================================
```

```
        private void LoadGroupsCombo()
        {
            using (var conn = DatabaseHelper.GetConnection())
            {
                conn.Open();
                var cmd = new SqlCommand("select GroupID, GroupName from StudentsGroups", conn);
                var reader = cmd.ExecuteReader();
                cbGroup.Items.Clear();
                while (reader.Read())
                {
                    cbGroup.Items.Add(new { ID = reader["GroupID"], Name =
reader["GroupName"].ToString() });
                }
                reader.Close();
            }
            cbGroup.DisplayMember = "Name";
            cbGroup.ValueMember = "ID";
        }

        private void LoadLessonsCombo()
        {
            using (var conn = DatabaseHelper.GetConnection())
            {
                conn.Open();
                var cmd = new SqlCommand("select LessonID, LessonName from Lessons", conn);
                var reader = cmd.ExecuteReader();
                cbLesson.Items.Clear();
                while (reader.Read())
```

```
            {
                cbLesson.Items.Add(new { ID = reader["LessonID"], Name =
reader["LessonName"].ToString() });
            }
            reader.Close();
        }
        cbLesson.DisplayMember = "Name";
        cbLesson.ValueMember = "ID";
    }

    private void LoadGroupLessonsGrid()
    {
        using (var conn = DatabaseHelper.GetConnection())
        {
            conn.Open();
            var cmd = new SqlCommand(@"
                select gl.GroupLessonID, g.GroupName as [Group Name], l.LessonName as [Lesson
Name], gl.WeeklyCount as [Weekly Hours]
                from GroupLessons gl
                join StudentsGroups g on gl.GroupID = g.GroupID
                join Lessons l on gl.LessonID = l.LessonID", conn);
            var adapter = new SqlDataAdapter(cmd);
            var table = new DataTable();
            adapter.Fill(table);
            dgvGroupLessons.DataSource = table;
        }
    }
```

```
        private void btnAddGroupLesson_Click(object sender, EventArgs e)
        {
            if (cbGroup.SelectedItem == null || cbLesson.SelectedItem == null || numWeeklyCount.Value
<= 0)
            {
                MessageBox.Show("Please select Group, Lesson, and set Weekly Hours greater than 0!");
                return;
            }

            dynamic selectedGroup = cbGroup.SelectedItem;
            dynamic selectedLesson = cbLesson.SelectedItem;

            using (var conn = DatabaseHelper.GetConnection())
            {
                conn.Open();
                var cmd = new SqlCommand("insert into GroupLessons (WeeklyCount, LessonID, GroupID)
values (@WeeklyCount, @LessonID, @GroupID)", conn);
                cmd.Parameters.AddWithValue("@WeeklyCount", (int)numWeeklyCount.Value);
                cmd.Parameters.AddWithValue("@LessonID", selectedLesson.ID);
                cmd.Parameters.AddWithValue("@GroupID", selectedGroup.ID);
                cmd.ExecuteNonQuery();

                MessageBox.Show("Lesson assigned to group successfully!");
                cbGroup.SelectedIndex = -1;
                cbLesson.SelectedIndex = -1;
                numWeeklyCount.Value = 0;

                LoadGroupLessonsGrid();
```

```
        }
    }

    private void btnDeleteGroupLesson_Click(object sender, EventArgs e)
    {
        if (dgvGroupLessons.SelectedRows.Count == 0)
        {
            MessageBox.Show("Please select an assignment to remove.");
            return;
        }

        int groupLessonID =
Convert.ToInt32(dgvGroupLessons.SelectedRows[0].Cells["GroupLessonID"].Value);

        try
        {
            using (var conn = DatabaseHelper.GetConnection())
            {
                conn.Open();
                var cmd = new SqlCommand("delete from GroupLessons where GroupLessonID = @ID",
conn);
                cmd.Parameters.AddWithValue("@ID", groupLessonID);
                cmd.ExecuteNonQuery();

                MessageBox.Show("Assignment removed successfully.");
                LoadGroupLessonsGrid();
            }
        }
```

```
    catch (SqlException ex)
    {
        MessageBox.Show("Error: " + ex.Message);
    }
}



private void label5_Click(object sender, EventArgs e)
{

}

private void label3_Click(object sender, EventArgs e)
{

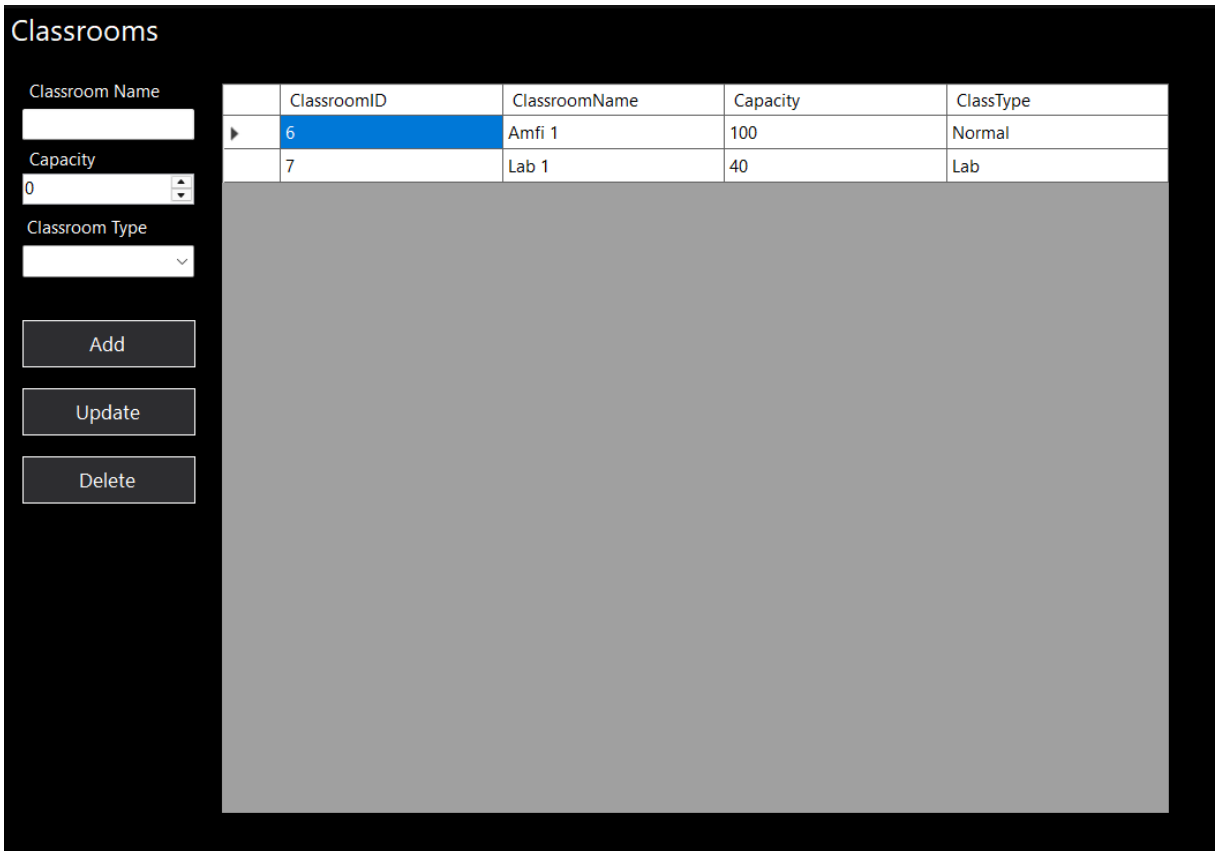
}

private void label6_Click(object sender, EventArgs e)
{}

private void dataGridView2_CellContentClick(object sender, DataGridViewCellEventArgs e)
{}

private void tabPage1_Click(object sender, EventArgs e)
{}}}
```

CLASSROOMS FORMU EKRAN FOTOĞRAFI:



CLASSROOMS FORMU KODLARI:

```
using MetroFramework.Forms;
using Microsoft.Data.SqlClient;
using System;
using System.Data;
using System.Windows.Forms;
using static System.Windows.Forms.VisualStyles.VisualStyleElement;

namespace CourseSchedule
{
```

```
public partial class ClassroomsControl : UserControl
{
    public ClassroomsControl()
    {
        InitializeComponent();
        LoadClassroomTypes();
        LoadClassrooms();
    }

    private void ClassroomControl_Load(object sender, EventArgs e)
    {
        LoadClassroomTypes();
        LoadClassrooms();
    }

    private void LoadClassroomTypes()
    {
        TypeCombo.Items.Clear();
        TypeCombo.Items.Add("Normal");
        TypeCombo.Items.Add("Lab");
    }

    private void LoadClassrooms()
    {
        using (var conn = DatabaseHelper.GetConnection())
        {
            conn.Open();
```

```
            var cmd = new SqlCommand("select * from Classrooms", conn);
            var adapter = new SqlDataAdapter(cmd);
            var table = new DataTable();
            adapter.Fill(table);
            dataGridView1.DataSource = table;
        }
    }

    private void Add_Button_Click(object sender, EventArgs e)
    {
        if (ClassroomName.Text == "" || TypeCombo.SelectedItem == null)
        {
            MessageBox.Show("Please fill all fields!");
            return;
        }

        int capacity = (int)CapacityNumberic.Value;
        string classType = TypeCombo.SelectedItem?.ToString() ?? "";

        using (var conn = DatabaseHelper.GetConnection())
        {
            conn.Open();
            var cmd = new SqlCommand(
                "insert into Classrooms (ClassroomName, Capacity, ClassType) values
(@ClassroomName, @Capacity, @ClassType)", conn);
            cmd.Parameters.AddWithValue("@ClassroomName", ClassroomName.Text);
            cmd.Parameters.AddWithValue("@Capacity", capacity);
            cmd.Parameters.AddWithValue("@ClassType", classType);
```

```
            cmd.ExecuteNonQuery();
            MessageBox.Show("Classroom added successfully!");

            ClassroomName.Text = "";
            CapacityNumberic.Value = 0;
            TypeCombo.SelectedIndex = -1;
            LoadClassrooms();
        }
    }

    private void Update_Button_Click(object sender, EventArgs e)
    {
        if (dataGridView1.SelectedRows.Count == 0)
        {
            MessageBox.Show("Please select a classroom to update.");
            return;
        }

        if (ClassroomName.Text == "" || TypeCombo.SelectedItem == null)
        {
            MessageBox.Show("Please fill all fields!");
            return;
        }

        int ClassroomID =
Convert.ToInt32(dataGridView1.SelectedRows[0].Cells["ClassroomID"].Value);

        using (var conn = DatabaseHelper.GetConnection())
```

```
        {
            conn.Open();
            var cmd = new SqlCommand(
                "update Classrooms set ClassroomName = @ClassroomName, Capacity = @Capacity,
ClassType = @ClassType where ClassroomID = @ClassroomID", conn);
            cmd.Parameters.AddWithValue("@ClassroomName", ClassroomName.Text);
            cmd.Parameters.AddWithValue("@Capacity", (int)CapacityNumberic.Value);
            cmd.Parameters.AddWithValue("@ClassType", TypeCombo.SelectedItem.ToString());
            cmd.Parameters.AddWithValue("@ClassroomID", ClassroomID);
            cmd.ExecuteNonQuery();
            MessageBox.Show("Classroom updated successfully.");

            ClassroomName.Text = "";
            CapacityNumberic.Value = 0;
            TypeCombo.SelectedIndex = -1;
            LoadClassrooms();
        }
    }

    private void Delete_Button_Click(object sender, EventArgs e)
    {
        if (dataGridView1.SelectedRows.Count == 0)
        {
            MessageBox.Show("Please select a classroom to delete.");
            return;
        }
```

```
        int ClassroomID =
Convert.ToInt32(dataGridView1.SelectedRows[0].Cells["ClassroomID"].Value);

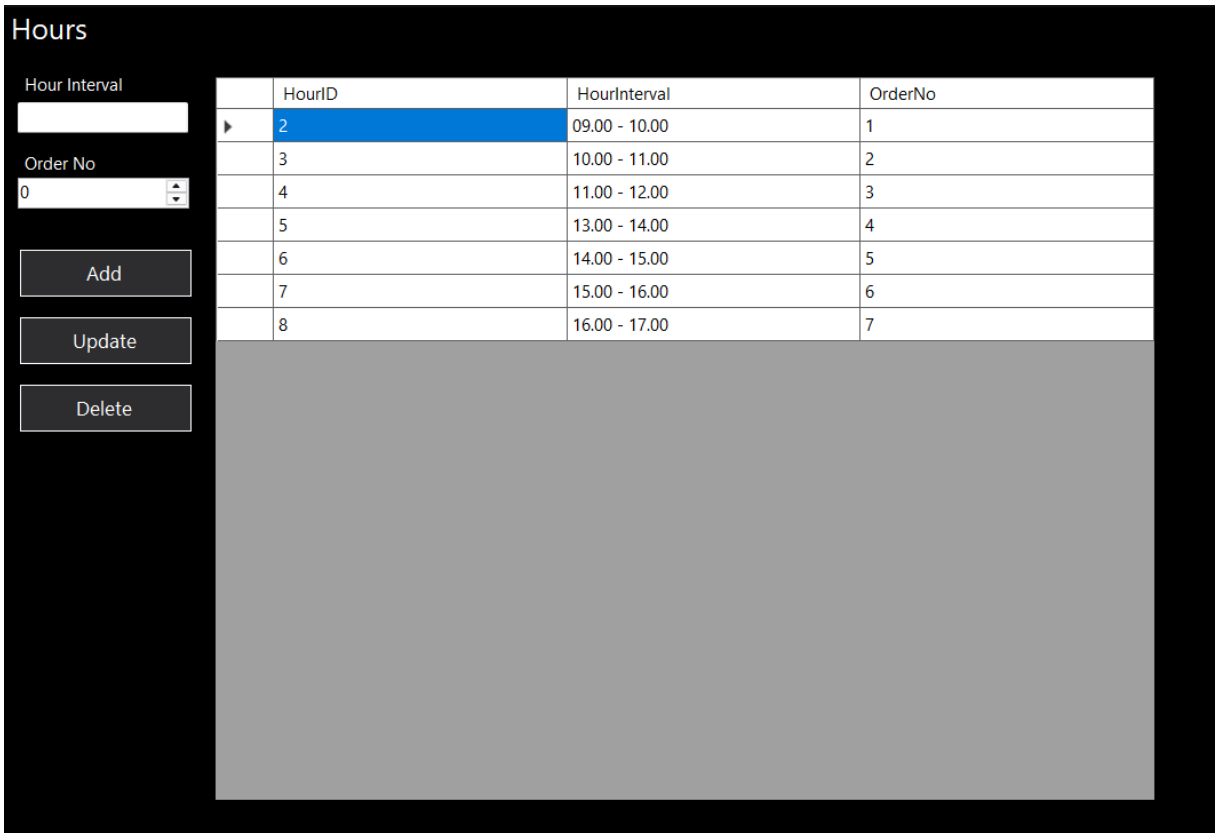

        try
        {
            using (var conn = DatabaseHelper.GetConnection())
            {
                conn.Open();
                var cmd = new SqlCommand(
                    "delete from Classrooms where ClassroomID = @ClassroomID", conn);
                cmd.Parameters.AddWithValue("@ClassroomID", ClassroomID);
                cmd.ExecuteNonQuery();
                MessageBox.Show("Classroom deleted successfully.");
                LoadClassrooms();
            }
        }
        catch (SqlException ex)
        {
            if (ex.Number == 547) // SQL Foreign Key (İlişki) Hatası
            {
                MessageBox.Show("Cannot delete! This classroom is currently assigned to a lesson in
the schedule. Please remove it from the schedule first.", "Delete Blocked", MessageBoxButtons.OK,
MessageBoxIcon.Warning);
            }
            else
            {
                MessageBox.Show("Database Error: " + ex.Message, "Error", MessageBoxButtons.OK,
MessageBoxIcon.Error);
            }
```

```
            }
        }

        private void ClassroomsControl_Load(object sender, EventArgs e)
        {

        }
    }
}
```

HOURS FORMU EKRAN FOTOĞRAFLARI:



HOURS FORMU KODLARI:

```
using Microsoft.Data.SqlClient;
using System;
using System.Data;
using System.Windows.Forms;

namespace CourseSchedule
{
    public partial class HourIntervalControl : UserControl
    {
        public HourIntervalControl()
```

```
{
    InitializeComponent();
    LoadHours();
}

private void LoadHours()
{
    using (var conn = DatabaseHelper.GetConnection())
    {
        conn.Open();
        var cmd = new SqlCommand("SELECT * FROM tbl_Hours ORDER BY OrderNo", conn);
        var adapter = new SqlDataAdapter(cmd);
        var table = new DataTable();
        adapter.Fill(table);
        dataGridView1.DataSource = table;
    }
}

private void Add_Button_Click(object sender, EventArgs e)
{
    if (string.IsNullOrWhiteSpace(hourIntervalTextBox.Text))
    {
        MessageBox.Show("Please enter hour interval!");
        return;
    }

    using (var conn = DatabaseHelper.GetConnection())
```

```
        {
            conn.Open();
            var cmd = new SqlCommand(
                "INSERT INTO tbl_Hours (HourInterval, OrderNo) VALUES (@HourInterval, @OrderNo)",
conn);
            cmd.Parameters.AddWithValue("@HourInterval", hourIntervalTextBox.Text);
            cmd.Parameters.AddWithValue("@OrderNo", (int)orderNoNumeric.Value);
            cmd.ExecuteNonQuery();
            MessageBox.Show("Hour added successfully!");
            hourIntervalTextBox.Clear();
            orderNoNumeric.Value = 0;
            LoadHours();
        }
    }

    private void Update_Button_Click(object sender, EventArgs e)
    {
        if (dataGridView1.SelectedRows.Count == 0)
        {
            MessageBox.Show("Please select an hour to update.");
            return;
        }

        if (string.IsNullOrWhiteSpace(hourIntervalTextBox.Text))
        {
            MessageBox.Show("Please enter hour interval!");
            return;
        }
```

```
        int hourID = Convert.ToInt32(dataGridView1.SelectedRows[0].Cells["HourID"].Value);

        using (var conn = DatabaseHelper.GetConnection())
        {
            conn.Open();
            var cmd = new SqlCommand(
                "UPDATE tbl_Hours SET HourInterval = @HourInterval, OrderNo = @OrderNo WHERE
HourID = @HourID", conn);
            cmd.Parameters.AddWithValue("@HourInterval", hourIntervalTextBox.Text);
            cmd.Parameters.AddWithValue("@OrderNo", (int)orderNoNumeric.Value);
            cmd.Parameters.AddWithValue("@HourID", hourID);
            cmd.ExecuteNonQuery();
            MessageBox.Show("Hour updated successfully.");
            hourIntervalTextBox.Clear();
            orderNoNumeric.Value = 0;
            LoadHours();
        }
    }

    private void Delete_Button_Click(object sender, EventArgs e)
    {
        if (dataGridView1.SelectedRows.Count == 0)
        {
            MessageBox.Show("Please select an hour to delete.");
            return;
        }
```

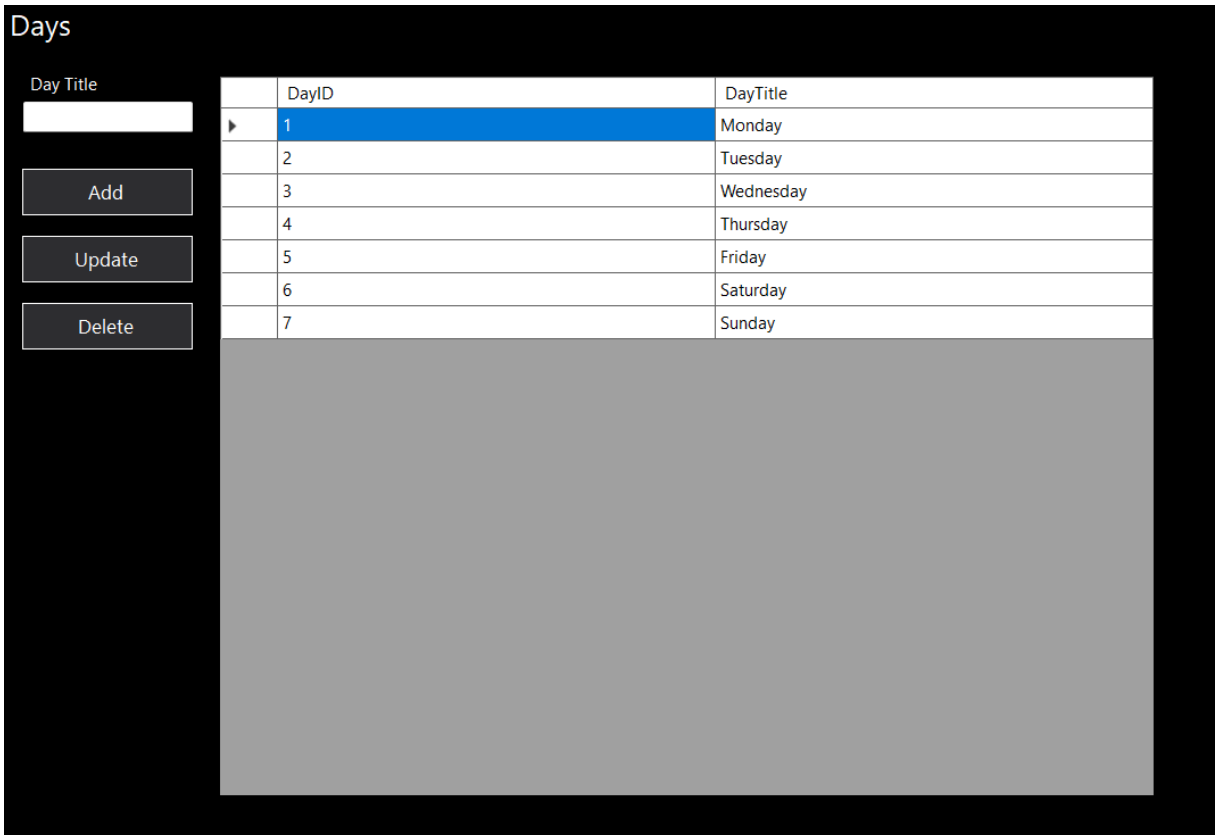
```
        int hourID = Convert.ToInt32(dataGridView1.SelectedRows[0].Cells["HourID"].Value);

        try
        {
            using (var conn = DatabaseHelper.GetConnection())
            {
                conn.Open();
                var cmd = new SqlCommand("DELETE FROM tbl_Hours WHERE HourID = @HourID",
conn);
                cmd.Parameters.AddWithValue("@HourID", hourID);
                cmd.ExecuteNonQuery();
                MessageBox.Show("Hour deleted successfully.");
                LoadHours();
            }
        }
        catch (SqlException ex)
        {
            if (ex.Number == 547)
            {
                MessageBox.Show("Cannot delete! This hour is used in schedules.");
            }
            else
            {
                MessageBox.Show("Error: " + ex.Message);
            }
        }
    }
```

```
        private void HourIntervalControl_Load(object sender, EventArgs e)
        {

        }
    }
}
```

DAYS FORMU EKRAN FOTOĞRAFI:



DAYS FORMU KODLARI:

```
using Microsoft.Data.SqlClient;
using System;
using System.Data;
using System.Windows.Forms;

namespace CourseSchedule
{
    public partial class DaysControl : UserControl
    {
        public DaysControl()
```

```
{
    InitializeComponent();
    LoadDays();
}

private void LoadDays()
{
    using (var conn = DatabaseHelper.GetConnection())
    {
        conn.Open();
        var cmd = new SqlCommand("SELECT * FROM tbl_Days", conn);
        var adapter = new SqlDataAdapter(cmd);
        var table = new DataTable();
        adapter.Fill(table);
        dataGridView1.DataSource = table;
    }
}

private void Add_Button_Click(object sender, EventArgs e)
{
    if (string.IsNullOrWhiteSpace(dayTitleTextBox.Text))
    {
        MessageBox.Show("Please enter day title!");
        return;
    }

    using (var conn = DatabaseHelper.GetConnection())
```

```
    {
        conn.Open();
        var cmd = new SqlCommand(
            "INSERT INTO tbl_Days (DayTitle) VALUES (@DayTitle)", conn);
        cmd.Parameters.AddWithValue("@DayTitle", dayTitleTextBox.Text);
        cmd.ExecuteNonQuery();
        MessageBox.Show("Day added successfully!");
        dayTitleTextBox.Clear();
        LoadDays();
    }
}

private void Update_Button_Click(object sender, EventArgs e)
{
    if (dataGridView1.SelectedRows.Count == 0)
    {
        MessageBox.Show("Please select a day to update.");
        return;
    }

    if (string.IsNullOrWhiteSpace(dayTitleTextBox.Text))
    {
        MessageBox.Show("Please enter day title!");
        return;
    }

    int dayID = Convert.ToInt32(dataGridView1.SelectedRows[0].Cells["DayID"].Value);
```

```
    using (var conn = DatabaseHelper.GetConnection())
    {
        conn.Open();
        var cmd = new SqlCommand(
            "UPDATE tbl_Days SET DayTitle = @DayTitle WHERE DayID = @DayID", conn);
        cmd.Parameters.AddWithValue("@DayTitle", dayTitleTextBox.Text);
        cmd.Parameters.AddWithValue("@DayID", dayID);
        cmd.ExecuteNonQuery();
        MessageBox.Show("Day updated successfully.");
        dayTitleTextBox.Clear();
        LoadDays();
    }
}

private void Delete_Button_Click(object sender, EventArgs e)
{
    if (dataGridView1.SelectedRows.Count == 0)
    {
        MessageBox.Show("Please select a day to delete.");
        return;
    }

    int dayID = Convert.ToInt32(dataGridView1.SelectedRows[0].Cells["DayID"].Value);

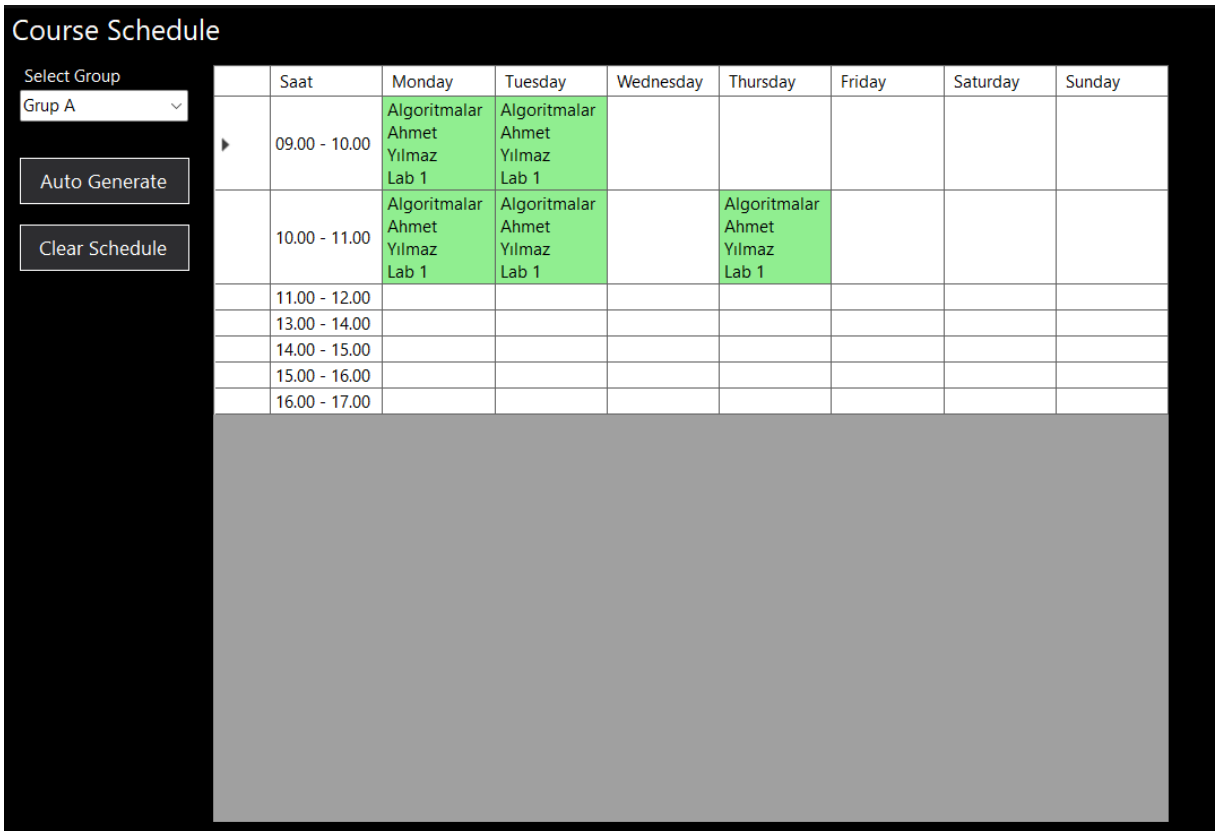
    try
    {
```

```
            using (var conn = DatabaseHelper.GetConnection())
            {
                conn.Open();
                var cmd = new SqlCommand("DELETE FROM tbl_Days WHERE DayID = @DayID",
conn);
                cmd.Parameters.AddWithValue("@DayID", dayID);
                cmd.ExecuteNonQuery();
                MessageBox.Show("Day deleted successfully.");
                LoadDays();
            }
        }
        catch (SqlException ex)
        {
            if (ex.Number == 547)
            {
                MessageBox.Show("Cannot delete! This day is used in schedules.");
            }
            else
            {
                MessageBox.Show("Error: " + ex.Message);
            }
        }
    }

    private void DaysControl_Load(object sender, EventArgs e)
    {


    }}}
```

COURSE SCHEDULE EKRAN FOTOĞRAFI:



COURSE SCHEDULE FORMU KODLARI:

```
using Microsoft.Data.SqlClient;
using System;
using System.Data;
using System.Drawing;
using System.Windows.Forms;

namespace CourseSchedule
{
    public partial class CourseScheduleControl : UserControl
    {
```

```
public CourseScheduleControl()
{
    InitializeComponent();

    cbScheduleGroup.SelectedIndexChanged += cbScheduleGroup_SelectedIndexChanged;

    // DataGridView Görsel Ayarları (Hücre içi metinler alt alta insin diye)
    dgvSchedule.DefaultCellStyle.WrapMode = DataGridViewTriState.True;
    dgvSchedule.AutoSizeRowsMode = DataGridViewAutoSizeRowsMode.AllCells;
    dgvSchedule.ReadOnly = true;
    dgvSchedule.AllowUserToAddRows = false;

    LoadGroupsCombo();
    InitializeGrid();
}

// ==========================================================================
// YÜKLEME VE ARAYÜZ
// ==========================================================================

private void LoadGroupsCombo()
{
    using (var conn = DatabaseHelper.GetConnection())
    {
        conn.Open();
        var cmd = new SqlCommand("select GroupID, GroupName from StudentsGroups", conn);
        var reader = cmd.ExecuteReader();
```

```
            cbScheduleGroup.Items.Clear();
            while (reader.Read())
            {
                cbScheduleGroup.Items.Add(new { ID = reader["GroupID"], Name =
reader["GroupName"].ToString() });
            }
        }
        cbScheduleGroup.DisplayMember = "Name";
        cbScheduleGroup.ValueMember = "ID";
    }

    private void InitializeGrid()
    {
        dgvSchedule.Columns.Clear();
        dgvSchedule.Rows.Clear();

        // İlk Kolon: Saatler
        dgvSchedule.Columns.Add("Col_Hour", "Saat");

        // Dinamik olarak günleri çekip kolon yapıyoruz
        using (var conn = DatabaseHelper.GetConnection())
        {
            conn.Open();
            var cmdDays = new SqlCommand("select DayID, DayTitle from tbl_Days order by DayID",
conn);
            var readerDays = cmdDays.ExecuteReader();
            while (readerDays.Read())
            {
```

```
            string colName = "Day_" + readerDays["DayID"].ToString();
            string colTitle = readerDays["DayTitle"].ToString()!;
            dgvSchedule.Columns.Add(colName, colTitle);
        }
        readerDays.Close();

        // Saat aralıklarını ekliyoruz
        var cmdHours = new SqlCommand("select HourID, HourInterval from tbl_Hours order by
OrderNo", conn);
        var readerHours = cmdHours.ExecuteReader();
        while (readerHours.Read())
        {
            int rowIndex = dgvSchedule.Rows.Add();
            dgvSchedule.Rows[rowIndex].Cells[0].Value = readerHours["HourInterval"].ToString();
            dgvSchedule.Rows[rowIndex].Tag = readerHours["HourID"]; // Saat ID'sini satırın gizli
tag'inde tutuyoruz
        }
    }
}

private void cbScheduleGroup_SelectedIndexChanged(object? sender, EventArgs e)
{
    if (cbScheduleGroup.SelectedItem == null) return;

    dynamic selectedGroup = cbScheduleGroup.SelectedItem;
    int groupID = selectedGroup.ID;

    RefreshScheduleDisplay(groupID);
```

```
}

private void RefreshScheduleDisplay(int groupID)
{
    // Önce tablonun içini temizle
    foreach (DataGridViewRow row in dgvSchedule.Rows)
    {
        for (int i = 1; i < dgvSchedule.Columns.Count; i++)
        {
            row.Cells[i].Value = "";
            row.Cells[i].Style.BackColor = Color.White;
        }
    }

    // Seçili grubun ders programını veritabanından çek ve hücrelere yerleştir
    using (var conn = DatabaseHelper.GetConnection())
    {
        conn.Open();
        var cmd = new SqlCommand(@"
            select cs.DayID, cs.HourID, l.LessonName, t.NameSurname, c.ClassroomName
            from CourseSchedule cs
            join Lessons l on cs.LessonID = l.LessonID
            join Teachers t on cs.TeacherID = t.TeacherID
            join Classrooms c on cs.ClassroomID = c.ClassroomID
            where cs.GroupID = @GroupID", conn);

        cmd.Parameters.AddWithValue("@GroupID", groupID);
```

```
            var reader = cmd.ExecuteReader();


            while (reader.Read())
            {
                int dayID = Convert.ToInt32(reader["DayID"]);
                int hourID = Convert.ToInt32(reader["HourID"]);
                string cellText =
$"{reader["LessonName"]}\n{reader["NameSurname"]}\n{reader["ClassroomName"]}";


                // Doğru satır ve sütunu bulup yazdır
                string colName = "Day_" + dayID;
                foreach (DataGridViewRow row in dgvSchedule.Rows)
                {
                    if (row.Tag != null && (int)row.Tag == hourID)
                    {
                        row.Cells[colName].Value = cellText;
                        row.Cells[colName].Style.BackColor = Color.LightGreen; // Dolu hücreleri yeşil yap
                        break;
                    }
                }
            }
        }
        dgvSchedule.ClearSelection();
    }


    // ======================================================================
    // OTOMATİK DAĞITIM ALGORİTMASI
    // ======================================================================
```

```
private void btnAutoGenerate_Click(object sender, EventArgs e)
{
    Cursor = Cursors.WaitCursor;
    int placedLessonsCount = 0;

    try
    {
        using (var conn = DatabaseHelper.GetConnection())
        {
            conn.Open();

            var cmdPending = new SqlCommand(@"
                select gl.GroupID, gl.LessonID, gl.WeeklyCount, g.NumberofPeople, l.LessonType,
                    (select count(*) from CourseSchedule cs where cs.GroupID = gl.GroupID and
cs.LessonID = gl.LessonID) as ScheduledCount
                from GroupLessons gl
                join StudentsGroups g on gl.GroupID = g.GroupID
                join Lessons l on gl.LessonID = l.LessonID", conn);

            var adapter = new SqlDataAdapter(cmdPending);
            var dtPending = new DataTable();
            adapter.Fill(dtPending);

            var dtDays = GetDataTable("select DayID from tbl_Days order by DayID", conn);
            var dtHours = GetDataTable("select HourID from tbl_Hours order by OrderNo", conn);

            foreach (DataRow pRow in dtPending.Rows)
```

```
{
    int groupID = Convert.ToInt32(pRow["GroupID"]);
    int lessonID = Convert.ToInt32(pRow["LessonID"]);
    int reqCapacity = Convert.ToInt32(pRow["NumberofPeople"]);
    string reqType = pRow["LessonType"].ToString()!.Trim();

    int weeklyCount = Convert.ToInt32(pRow["WeeklyCount"]);
    int scheduledCount = Convert.ToInt32(pRow["ScheduledCount"]);
    int needed = weeklyCount - scheduledCount;

    for (int i = 0; i < needed; i++)
    {
        bool placed = false;
        foreach (DataRow dRow in dtDays.Rows)
        {
            foreach (DataRow hRow in dtHours.Rows)
            {
                int dayID = Convert.ToInt32(dRow["DayID"]);
                int hourID = Convert.ToInt32(hRow["HourID"]);

                // Günde Maksimum 2 Saat Aynı Ders
                var cmdFind = new SqlCommand(@"
                    select top 1 t.TeacherID, c.ClassroomID
                    from TeacherLessons tl
                    join Teachers t on tl.TeacherID = t.TeacherID
                    cross join Classrooms c
                    where tl.LessonID = @LessonID
```

```
                    and c.Capacity >= @ReqCapacity

                    and c.ClassType = @ReqType

                    and not exists (select 1 from CourseSchedule cs where cs.TeacherID =
t.TeacherID and cs.DayID = @DayID and cs.HourID = @HourID)

                    and not exists (select 1 from TeacherAvailability ta where ta.TeacherID =
t.TeacherID and ta.DayID = @DayID and ta.HourID = @HourID and ta.AvailabilityStatus = 0)

                    and not exists (select 1 from CourseSchedule cs where cs.ClassroomID =
c.ClassroomID and cs.DayID = @DayID and cs.HourID = @HourID)

                    and not exists (select 1 from CourseSchedule cs where cs.GroupID =
@GroupID and cs.DayID = @DayID and cs.HourID = @HourID)

                    and (select count(*) from CourseSchedule cs2 where cs2.GroupID =
@GroupID and cs2.LessonID = @LessonID and cs2.DayID = @DayID) < 2", conn);


                cmdFind.Parameters.AddWithValue("@LessonID", lessonID);

                cmdFind.Parameters.AddWithValue("@ReqCapacity", reqCapacity);

                cmdFind.Parameters.AddWithValue("@ReqType", reqType);

                cmdFind.Parameters.AddWithValue("@DayID", dayID);

                cmdFind.Parameters.AddWithValue("@HourID", hourID);

                cmdFind.Parameters.AddWithValue("@GroupID", groupID);


                var readerFind = cmdFind.ExecuteReader();

                if (readerFind.Read())

                {

                    int teacherID = Convert.ToInt32(readerFind["TeacherID"]);

                    int classroomID = Convert.ToInt32(readerFind["ClassroomID"]);

                    readerFind.Close();


                    var cmdInsert = new SqlCommand("insert into CourseSchedule (GroupID,
LessonID, HourID, DayID, ClassroomID, TeacherID, IsLocked, IsAutoGenerated) values (@GroupID,
@LessonID, @HourID, @DayID, @ClassroomID, @TeacherID, 0, 1)", conn);
```

```
                        cmdInsert.Parameters.AddWithValue("@GroupID", groupID);
                        cmdInsert.Parameters.AddWithValue("@LessonID", lessonID);
                        cmdInsert.Parameters.AddWithValue("@HourID", hourID);
                        cmdInsert.Parameters.AddWithValue("@DayID", dayID);
                        cmdInsert.Parameters.AddWithValue("@ClassroomID", classroomID);
                        cmdInsert.Parameters.AddWithValue("@TeacherID", teacherID);
                        cmdInsert.ExecuteNonQuery();

                        placed = true;
                        placedLessonsCount++;
                        break;
                    }
                    if (!readerFind.IsClosed) readerFind.Close();
                }
                if (placed) break;
            }
        }
    }
}

    MessageBox.Show($"Auto-Generation complete! Successfully placed
{placedLessonsCount} lessons into the schedule.", "Algorithm Result", MessageBoxButtons.OK,
MessageBoxIcon.Information);

    if (cbScheduleGroup.SelectedItem != null)
    {
        dynamic selectedGroup = cbScheduleGroup.SelectedItem;
        RefreshScheduleDisplay(selectedGroup.ID);
```

```
            }
        }
        catch (Exception ex)
        {
            MessageBox.Show("An error occurred during scheduling: " + ex.Message);
        }
        finally
        {
            Cursor = Cursors.Default;
        }
    }

    private void btnClearSchedule_Click(object sender, EventArgs e)
    {
        var result = MessageBox.Show("Are you sure you want to delete the ENTIRE schedule? This cannot be undone.", "Warning", MessageBoxButtons.YesNo, MessageBoxIcon.Warning);

        if (result == DialogResult.Yes)
        {
            using (var conn = DatabaseHelper.GetConnection())
            {
                conn.Open();
                var cmd = new SqlCommand("delete from CourseSchedule", conn);
                cmd.ExecuteNonQuery();
            }

            if (cbScheduleGroup.SelectedItem != null)
            {
```

```
            dynamic selectedGroup = cbScheduleGroup.SelectedItem;
            RefreshScheduleDisplay(selectedGroup.ID);
        }
        MessageBox.Show("Schedule cleared successfully.");
    }
}

// Yardımcı Metot
private DataTable GetDataTable(string query, SqlConnection conn)
{
    var cmd = new SqlCommand(query, conn);
    var adapter = new SqlDataAdapter(cmd);
    var dt = new DataTable();
    adapter.Fill(dt);
    return dt;
}

private void CourseScheduleControl_Load(object sender, EventArgs e)
{
    dgvSchedule.CellDoubleClick += dgvSchedule_CellDoubleClick;

    if (Form1.UserRole == "Student")
    {
        btnAutoGenerate.Visible = false;
        btnClearSchedule.Visible = false;

        dgvSchedule.CellDoubleClick -= dgvSchedule_CellDoubleClick;
```

```
    }
}
private void dgvSchedule_CellDoubleClick(object? sender, DataGridViewCellEventArgs e)
{
    // Başlıklara veya saat kolonuna tıklandıysa işlemi iptal et
    if (e.RowIndex < 0 || e.ColumnIndex <= 0) return;
    if (cbScheduleGroup.SelectedItem == null) return;

    dynamic selectedGroup = cbScheduleGroup.SelectedItem;
    int groupID = selectedGroup.ID;

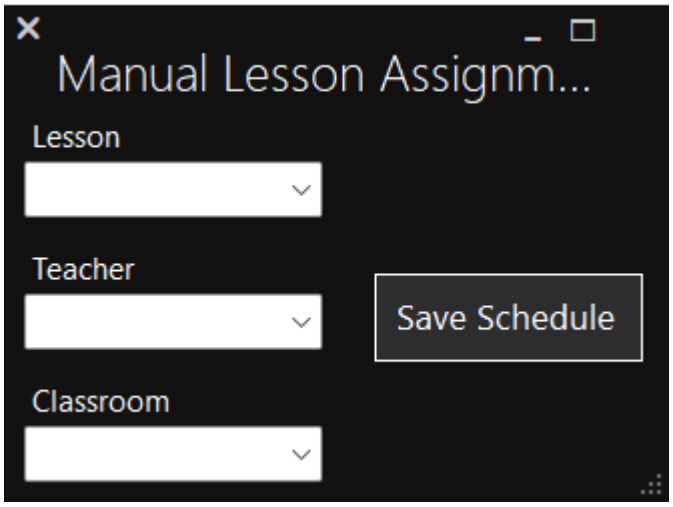
    int hourID = Convert.ToInt32(dgvSchedule.Rows[e.RowIndex].Tag);
    string colName = dgvSchedule.Columns[e.ColumnIndex].Name;
    int dayID = Convert.ToInt32(colName.Replace("Day_", ""));

    var cellValue = dgvSchedule.Rows[e.RowIndex].Cells[e.ColumnIndex].Value;

    // HÜCRE DOLUYSA (Silme İşlemi)
    if (cellValue != null && !string.IsNullOrWhiteSpace(cellValue.ToString()))
    {
        var result = MessageBox.Show("Are you sure you want to remove this lesson from the
schedule?", "Remove Lesson", MessageBoxButtons.YesNo, MessageBoxIcon.Question);
        if (result == DialogResult.Yes)
        {
            using (var conn = DatabaseHelper.GetConnection())
            {
                conn.Open();
```

```
                    var cmd = new SqlCommand("delete from CourseSchedule where GroupID =
@GroupID and DayID = @DayID and HourID = @HourID", conn);
                    cmd.Parameters.AddWithValue("@GroupID", groupID);
                    cmd.Parameters.AddWithValue("@DayID", dayID);
                    cmd.Parameters.AddWithValue("@HourID", hourID);
                    cmd.ExecuteNonQuery();
                }
                RefreshScheduleDisplay(groupID);
            }
        }
        // HÜCRE BOŞSA (Ekleme İşlemi)
        else
        {
            using (var manualForm = new ManualAssignForm(groupID, dayID, hourID))
            {
                if (manualForm.ShowDialog() == DialogResult.OK)
                {
                    RefreshScheduleDisplay(groupID);
                }
            }
        }
    }
}
```

MANUAL ASSİGN FORMU EKRAN FOTOĞRAFI:



MANUAL ASSİNG FORM KODLARI:

```
using MetroFramework.Forms;
using Microsoft.Data.SqlClient;
using System;
using System.Data;
using System.Windows.Forms;

namespace CourseSchedule
{
    public partial class ManualAssignForm : MetroForm
    {
        private int _groupID;
        private int _dayID;
        private int _hourID;

        public ManualAssignForm(int groupID, int dayID, int hourID)
        {
```

```
    InitializeComponent();
    _groupID = groupID;
    _dayID = dayID;
    _hourID = hourID;

    cbLessons.SelectedIndexChanged += cbLessons_SelectedIndexChanged;

    LoadLessons();
    LoadClassrooms();

    this.Text = "Manual Lesson Assignment";
    this.StartPosition = FormStartPosition.CenterParent;
}

private void LoadLessons()
{
    using (var conn = DatabaseHelper.GetConnection())
    {
        conn.Open();
        var cmd = new SqlCommand(@"
            select l.LessonID, l.LessonName
            from GroupLessons gl
            join Lessons l on gl.LessonID = l.LessonID
            where gl.GroupID = @GroupID", conn);
        cmd.Parameters.AddWithValue("@GroupID", _groupID);
        var reader = cmd.ExecuteReader();
        cbLessons.Items.Clear();
```

```
            while (reader.Read())
            {
                cbLessons.Items.Add(new { ID = reader["LessonID"], Name =
reader["LessonName"].ToString() });
            }
        }
        cbLessons.DisplayMember = "Name";
        cbLessons.ValueMember = "ID";
    }

    private void cbLessons_SelectedIndexChanged(object sender, EventArgs e)
    {
        if (cbLessons.SelectedItem == null) return;
        dynamic selectedLesson = cbLessons.SelectedItem;
        int lessonID = selectedLesson.ID;

        using (var conn = DatabaseHelper.GetConnection())
        {
            conn.Open();
            var cmd = new SqlCommand(@"
                select t.TeacherID, t.NameSurname
                from TeacherLessons tl
                join Teachers t on tl.TeacherID = t.TeacherID
                where tl.LessonID = @LessonID", conn);
            cmd.Parameters.AddWithValue("@LessonID", lessonID);
            var reader = cmd.ExecuteReader();
            cbTeachers.Items.Clear();
            while (reader.Read())
```

```
            {
                cbTeachers.Items.Add(new { ID = reader["TeacherID"], Name =
reader["NameSurname"].ToString() });
            }
        }
        cbTeachers.DisplayMember = "Name";
        cbTeachers.ValueMember = "ID";
    }

    private void LoadClassrooms()
    {
        using (var conn = DatabaseHelper.GetConnection())
        {
            conn.Open();
            var cmd = new SqlCommand("select ClassroomID, ClassroomName from Classrooms",
conn);
            var reader = cmd.ExecuteReader();
            cbClassrooms.Items.Clear();
            while (reader.Read())
            {
                cbClassrooms.Items.Add(new { ID = reader["ClassroomID"], Name =
reader["ClassroomName"].ToString() });
            }
        }
        cbClassrooms.DisplayMember = "Name";
        cbClassrooms.ValueMember = "ID";
    }

    private void btnSave_Click(object sender, EventArgs e)
```

```
    {
        if (cbLessons.SelectedItem == null || cbTeachers.SelectedItem == null ||
cbClassrooms.SelectedItem == null)
        {
            MessageBox.Show("Please make all selections!");
            return;
        }

        dynamic selectedLesson = cbLessons.SelectedItem;
        dynamic selectedTeacher = cbTeachers.SelectedItem;
        dynamic selectedClassroom = cbClassrooms.SelectedItem;

        int lessonID = selectedLesson.ID;
        int teacherID = selectedTeacher.ID;
        int classroomID = selectedClassroom.ID;

        using (var conn = DatabaseHelper.GetConnection())
        {
            conn.Open();

            var cmdCheck = new SqlCommand(@"
                select
                    (select count(*) from CourseSchedule where TeacherID = @TeacherID and DayID =
@DayID and HourID = @HourID) as TeacherConflict,
                    (select count(*) from TeacherAvailability where TeacherID = @TeacherID and DayID =
@DayID and HourID = @HourID and AvailabilityStatus = 0) as TeacherNotAvailable,
                    (select count(*) from CourseSchedule where ClassroomID = @ClassroomID and DayID
= @DayID and HourID = @HourID) as ClassroomConflict,
```

```
                (select Capacity from Classrooms where ClassroomID = @ClassroomID) as
ClassCapacity,
                (select NumberofPeople from StudentsGroups where GroupID = @GroupID) as
GroupSize", conn);


            cmdCheck.Parameters.AddWithValue("@TeacherID", teacherID);
            cmdCheck.Parameters.AddWithValue("@ClassroomID", classroomID);
            cmdCheck.Parameters.AddWithValue("@DayID", _dayID);
            cmdCheck.Parameters.AddWithValue("@HourID", _hourID);
            cmdCheck.Parameters.AddWithValue("@GroupID", _groupID);


            var reader = cmdCheck.ExecuteReader();
            if (reader.Read())
            {
               if (Convert.ToInt32(reader["TeacherConflict"]) > 0)
               {
                  MessageBox.Show("Error: Selected teacher is assigned to another class at this time!",
"Conflict", MessageBoxButtons.OK, MessageBoxIcon.Error);
                  reader.Close(); return;
               }
               if (Convert.ToInt32(reader["TeacherNotAvailable"]) > 0)
               {
                  MessageBox.Show("Error: Teacher is not available at this day/hour!", "Availability
Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
                  reader.Close(); return;
               }
               if (Convert.ToInt32(reader["ClassroomConflict"]) > 0)
               {
```

```
                    MessageBox.Show("Error: Selected classroom is full at this time!", "Classroom
Conflict", MessageBoxButtons.OK, MessageBoxIcon.Error);
                    reader.Close(); return;
                }
                if (Convert.ToInt32(reader["ClassCapacity"]) < Convert.ToInt32(reader["GroupSize"]))
                {
                    MessageBox.Show("Error: Classroom capacity is smaller than the group size!",
"Capacity Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
                    reader.Close(); return;
                }
            }
            reader.Close();


            var cmdInsert = new SqlCommand(@"
                insert into CourseSchedule (GroupID, LessonID, HourID, DayID, ClassroomID,
TeacherID, IsLocked, IsAutoGenerated)
                values (@GroupID, @LessonID, @HourID, @DayID, @ClassroomID, @TeacherID, 0,
0)", conn);


            cmdInsert.Parameters.AddWithValue("@GroupID", _groupID);
            cmdInsert.Parameters.AddWithValue("@LessonID", lessonID);
            cmdInsert.Parameters.AddWithValue("@HourID", _hourID);
            cmdInsert.Parameters.AddWithValue("@DayID", _dayID);
            cmdInsert.Parameters.AddWithValue("@ClassroomID", classroomID);
            cmdInsert.Parameters.AddWithValue("@TeacherID", teacherID);


            cmdInsert.ExecuteNonQuery();
            MessageBox.Show("Lesson manually assigned successfully!", "Success",
MessageBoxButtons.OK, MessageBoxIcon.Information);
```

```
            this.DialogResult = DialogResult.OK;
            this.Close();
        }
    }

    private void ManualAssignForm_Load(object sender, EventArgs e)
    {

    }
  }
}
```

DATABASEHELPER.CS KOD DİZESİ:

```
using System;
using System.Collections.Generic;
using System.Text;
using Microsoft.Data.SqlClient;

namespace CourseSchedule
{
    internal class DatabaseHelper
    {
        // Projedeki veritabanı bağlantı dizesi. Veritabanı sunucusu, veritabanı adı ve güvenlik bilgilerini içerir.
        private static string connectionString =
            "Server=localhost;Database=CourseSchedule;Integrated Security=True;TrustServerCertificate=True";

        public static SqlConnection GetConnection()
        {
            return new SqlConnection(connectionString);
        }
    }
}
```

SQL KODLARI:

```
create table Faculties(
        FacultyID int primary key identity(1,1),
        FacultyName nvarchar(100) not null)

create table Departments(
        DepartmentID int primary key identity(1,1),
        DepartmentName nvarchar(100) not null,
        FacultyID int not null,
    foreign key (FacultyID) references Faculties(FacultyID))

create table tbl_Hours(
        HourID int primary key identity(1,1),
        HourInterval nvarchar(100) not null,
        OrderNo int not null)

create table tbl_Days(
        DayID int primary key identity(1,1),
        DayTitle nvarchar(100) not null)

create table Classrooms(
        ClassroomID int primary key identity(1,1),
        ClassroomName nvarchar(100) not null,
        Capacity int not null,
        ClassType nvarchar(100) not null)
```

```
create table Teachers(
	TeacherID int primary key identity(1,1),
	DepartmentID int not null,
	NameSurname nvarchar(100) not null,
	Title nvarchar(100) not null,
	foreign key (DepartmentID) references Departments(DepartmentID))

create table Lessons(
	LessonID int primary key identity(1,1),
	Duration int not null,
	DepartmentID int not null,
	LessonType nvarchar(100) not null,
	LessonName nvarchar(100) not null,
	Credit int not null,
	foreign key (DepartmentID) references Departments(DepartmentID))

create table StudentsGroups(
	GroupID int primary key identity(1,1),
	GroupName nvarchar(100) not null,
	NumberofPeople int not null,
	DepartmentID int not null,
	foreign key (DepartmentID) references Departments(DepartmentID))

create table TeacherLessons(
	TeacherLessonID int primary key identity(1,1),
	TeacherID int not null,
	LessonID int not null,
```

```
    foreign key (TeacherID) references Teachers(TeacherID),
    foreign key (LessonID) references Lessons(LessonID))

create table TeacherAvailability(
    UnID int primary key identity(1,1),
    TeacherID int not null,
    DayID int not null,
    HourID int not null,
    AvailabilityStatus bit not null,
    foreign key (TeacherID) references Teachers(TeacherID),
    foreign key (DayID) references tbl_Days(DayID),
    foreign key (HourID) references tbl_Hours(HourID))

create table GroupLessons(
    GroupLessonID int primary key identity(1,1),
    WeeklyCount int not null,
    LessonID int not null,
    GroupID int not null,
    foreign key (LessonID) references Lessons(LessonID),
    foreign key (GroupID) references StudentsGroups(GroupID))

create table CourseSchedule(
    ScheduleID int primary key identity(1,1),
    GroupID int not null,
    LessonID int not null,
    HourID int not null,
    DayID int not null,
```

```
ClassroomID int not null,
TeacherID int not null,
IsLocked bit not null default 0,
IsAutoGenerated bit not null default 0,
foreign key (GroupID) references StudentsGroups(GroupID),
foreign key (LessonID) references Lessons(LessonID),
foreign key (HourID) references tbl_Hours(HourID),
foreign key (DayID) references tbl_Days(DayID),
foreign key (TeacherID) references Teachers(TeacherID),
foreign key (ClassroomID) references Classrooms(ClassroomID))
```